

Hybrid genetic algorithms for minimizing makespan in dynamic job shop scheduling problem[☆]



Nilsen Kundakcı^a, Osman Kulak^{b,*}

^a Department of Business Administration, Pamukkale University, 20070 Denizli, Turkey

^b Department of Industrial Engineering, Pamukkale University, 20070 Denizli, Turkey

ARTICLE INFO

Article history:

Received 21 February 2015

Received in revised form 17 August 2015

Accepted 10 March 2016

Available online 22 March 2016

Keywords:

Hybrid genetic algorithm

Tabu search

Dynamic job shop scheduling

ABSTRACT

Job shop scheduling has been the focus of a substantial amount of research over the last decade and most of these approaches are formulated and designed to address the static job shop scheduling problem. Dynamic events such as random job arrivals, machine breakdowns and changes in processing time, which are inevitable occurrences in production environment, are ignored in static job shop scheduling problem. As dynamic job shop scheduling problem is known NP-hard combinatorial optimization, this paper introduces efficient hybrid Genetic Algorithm (GA) methodologies for minimizing makespan in this kind of problem. Various benchmark problems including the number of jobs, the number of machines, and different dynamic events are generated and detailed numerical experiments are carried out to evaluate the performance of proposed methodologies. The numerical results indicate that the proposed methods produce superior solutions for well-known benchmark problems compared to those reported in the literature.

© 2016 Elsevier Ltd. All rights reserved.

1. Introduction

Effective production is very important in today's competitive environment. Companies are under pressure to shorten their lead times and meet customer requirements to maintain customer satisfaction. Production planning and scheduling plays a crucial role in increasing the efficiency of the plants. Scheduling is a decision making process that concerns the allocation of limited resources among competing tasks over time with the goal of optimizing one or more objectives (Pinedo, 2008). Proper scheduling leads to increased efficiency and capacity utilization, reduced time required to complete tasks, and consequently, increased profitability for the organization (Vinod & Sridharan, 2008).

Most scheduling problems are defined as complex combinatorial optimization problems. Job shop scheduling is a branch of production scheduling, which is among the most challenging combinatorial optimization problems (Motaghedi-larijani, Sabri-laghaie, & Heydar, 2010). The job shop scheduling (JSS) problem involves determining a schedule for jobs that have pre-specified operation sequences in a multi-machine environment (Fattahi, Mehrabad, & Jolai, 2007). A JSS problem generally refers to a static

scheduling problem in the job shop, where a set of different jobs are processed on a series of machines. Each job has some operations that must be processed in a certain technical sequence and each operation has a predetermined processing time. The main goal of a JSS problem is to find a schedule to process all jobs in a manner that optimizes given performance objectives (Zhou, Nee, & Lee, 2009). The general JSS problem is strongly Non-deterministic Polynomial-time hard (NP-hard). The JSS problem shifts to a new kind of problem that is known as a dynamic job shop scheduling (DJSS) problem when real time events such as random job arrivals, machine breakdowns and changes in processing times occur (Adibi, Zandieh, & Amiri, 2010). DJSS is also NP-hard combinatorial optimization problem. In the scheduling literature, most publications focus on static scheduling problems and seldom take into account dynamic factors. However in real life, scheduling problems that naturally include dynamic events are more complex than the job scheduling problem (JSP).

Although there are several research papers in the literature which address these problems, there is potential for developing fast and effective solutions. The main contribution of this paper is to develop different hybrid GA approaches for dynamic job shop scheduling problems to obtain efficient solutions in terms of solution quality and computational efficiency. Firstly, we develop a new KK (Kundakcı & Kulak) heuristic and it is combined with GA for dynamic job shop scheduling problems including new job

[☆] This manuscript was processed by Area Editor 'Mitsuo Gen'.

* Corresponding author. Tel.: +90 258 2963151.

E-mail addresses: nilsenk@pau.edu.tr (N. Kundakcı), okulak@pau.edu.tr (O. Kulak).

arrivals, machine breakdowns and changes in the processing time. Moreover, well-known dispatching rules are integrated with GA to obtain outstanding initial population. In addition, various benchmark problems including the number of jobs, the number of machines, and different dynamic events are generated and presented in the paper to evaluate the performances of the proposed methodologies. Finally, we provide a numerical evaluation, which demonstrates the superiority of the hybrid GA approaches over conventional GA and tabu search (TS) algorithms.

This study is organized as follows: In the second section, a comprehensive literature review of dynamic job shop scheduling is given and then, a mathematical model of the problem is presented. Next, efficient heuristic solution procedures are proposed. Experiments are conducted to demonstrate the effectiveness of the proposed methods and detailed numerical results are presented in the fifth section. Finally, results of the experiments are discussed and suggestions for future research are offered.

2. Literature review

The job shop scheduling problem is well-known to be one of the hardest combinatorial optimization problems. JSS problems have captured the interest of many researchers. Although there has been considerable interest in JSS problems, several research papers have been published that consider dynamic factors. Dynamic job shop scheduling problems are also combinatorial in nature and known to be NP-hard. In the literature, various heuristic methods have been proposed to obtain a near optimum solution. The first study in dynamic job shop scheduling was published by Holloway and Nelson in 1974. They developed a multi-pass heuristic scheduling procedure for job shop scheduling problems with due dates and variable processing times. Another study that used the same method for the scheduling of a job shop with job due dates, intermittent job arrivals, and statistical processing times was conducted by Nelson et al. in 1977.

Other heuristic methods have also been proposed for dynamic job shop scheduling problems. Muhlemann, Lockett, and Farn (1982) used heuristics in the form of dispatching disciplines for the dynamic scheduling problem that includes new job arrivals and uncertainty like errors in estimating process times and machine breakdowns. Their experiments indicated that more frequent revisions are needed to obtain better scheduling performance. Chang (1997) developed a new approach to provide real-time estimates of queuing times for the remaining operations of jobs in the dynamic job shop and incorporated this estimated queuing time information into existing scheduling heuristics to improve their performance.

Another paper that considers dispatching rules in dynamic job shop scheduling problems is from Rajendran and Holthaus (1999). They presented a comparative study on the performance of dispatching rules in dynamic flow shops, job shops and flow shops, and job shops with missing operations and they observed that the performance of dispatching rules is being influenced by the routing of jobs and shop floor configurations. Dominic, Kaliyamoorthy, and Kumar (2004) proposed an efficient method for dynamic job shop scheduling by combining different dispatching rules and simulated the dynamic model of a job shop under various rules and performance measures. Vinod and Sridharan (2008) presented a simulation-based experimental study of dispatching rules for scheduling a dynamic job shop in which the setup times are sequence dependent and experiment results indicated that setup-oriented rules provide better performance than ordinary rules. Lu and Romanowski (2013) proposed multicontextual dispatching rules for job shops with dynamic job arrival. They examined 11 basic dispatching rules and 33 composite rules made

with multi-contextual functions that describe machine idle time and job waiting time. Sharma and Jain (2014) assessed the performance of nine dispatching rules in dynamic job shop from make-span, mean flow time, maximum flow time, mean tardiness, maximum tardiness, number of tardy jobs, total setups and mean setup time performance measures viewpoint. Results indicated that shortest setup time rule provides the best performance for mean flow time and number of tardy jobs measures.

In the literature, different authors propose reactive scheduling policies for dynamic job shop problems. Sabuncuoğlu and Bayiz (2000) studied the reactive scheduling problems in a stochastic manufacturing environment and tested several scheduling policies under machine breakdown in a classical job shop system. Kutanoğlu and Sabuncuoğlu (2001) developed reactive scheduling policies against unexpected machine failures in dynamic job shop problems. Suwa and Sandoh (2007) proposed a new when-to-schedule policy in reactive scheduling and demonstrated the effectiveness of the proposed policy by applying it to job shop problems with machine breakdown. Nie, Gao, Li, and Shao (2013) proposed a heuristic to implement the reactive scheduling of the jobs in the dynamic production environment. They also proposed an approach based on gene expression programming (GEP) to construct efficient scheduling rules for dynamic job shop scheduling.

Genetic Algorithm (GA) has frequently been used for dynamic job shop scheduling problems. Qi, Burns, and Harrison (2000) used parallel multi-population GA to meet the dynamic nature of the job-shop scheduling. They compared the results of the proposed GA with dispatching rules and proved that the proposed GA successfully improved the solution. Rangsaritratsamee, Ferrell, and Kurz (2004) proposed a methodology for dynamic job shop scheduling that simultaneously addresses efficiency and stability though a multi-objective approach. Their methodology uses periodic rescheduling in which a multi-criteria objective function is used as the fitness function of a GA to generate the schedules at each rescheduling point. They tested the methodology on a simulated job shop to determine the impact of the key parameters on the performance measures. Fattahi and Fallahi (2010) developed a meta-heuristic algorithm based on a GA for dynamic scheduling in a flexible job shop and considered two objectives to balance efficiency and stability in the schedules. Kapanoglu and Alikalfa (2011) proposed a GA based learning system for constructing interval-based, state-dependent priority rules for each interval of queue lengths in dynamic job shops and their proposed approach considerably outperformed the priority rules for most of the problems.

Meta-heuristic methods based on local search are also used in dynamic job shop scheduling problems. Liu, Ong, and Ng (2005) proposed Tabu Search for solving the static shop scheduling problem for a number of dynamic shop scheduling benchmark problems and analyzed the characteristics of the dynamic shop scheduling problem when machine breakdowns and new job arrivals occur. Hosseinabadi, Siar, Shamshirband, Shojafar, and Nasir (2014) developed a new method called TIME_GELS that uses the gravitational emulation local search algorithm (GELS) for solving the multi-objective flexible dynamic job-shop scheduling problem. Also evolutionary algorithms are proposed for dynamic job shop scheduling problems. Hao and Lin (2010) focused on a job shop rescheduling problem in dynamic environment which considers random job arrivals and machine breakdowns and proposed an interactive adaptive-weight evolutionary algorithm. Hao and Gen (2011) formulated a multi-objective job shop rescheduling problem to improve the practical application of rescheduling. They designed an evolutionary algorithm in which a random key-based representation and interactive adaptive-weight fitness assignment are embedded.

Furthermore, hybrid methods are proposed to solve dynamic job shop scheduling problems. Li and Chen (2009) analyzed the

Table 1

Summary of dynamic job shop scheduling applications in the literature.

Authors	Dynamic factors	Method
Holloway and Nelson (1974)	Variable processing times	Multi-pass heuristic scheduling procedure
Nelson, Holloway, and Wong (1977)	Intermittent job arrivals, statistical processing times	Multi-pass heuristic scheduling procedure
Muhlemann et al. (1982)	Intermittent job arrivals, uncertainty in estimating processing times, machine breakdown	Heuristic methods
Chang (1997)	New job arrivals	Heuristic methods
Rajendran and Holthaus (1999)	Dynamic job arrivals	Dispatching rules
Qi et al. (2000)	New job arrivals	Parallel multi-population genetic algorithm
Sabuncuoğlu and Bayız (2000)	Machine breakdowns	Reactive scheduling
Kutanoglu and Sabuncuoğlu (2001)	Unexpected machine breakdown	Reactive scheduling
Dominic et al. (2004)	New job arrivals	A combination of dispatching rules
Rangaritratsamee et al. (2004)	Dynamic job arrivals	Genetic local search
Liu et al. (2005)	Machine breakdown and new job arrivals	Tabu search
Suwa and Sandoh (2007)	Machine breakdown	Reactive scheduling
Vinod and Sridharan (2008)	Sequence dependent setups	New setup-oriented dispatching rules
Gao et al. (2009)	Machine breakdown and cancellation of order	Hybrid method based on ant-colony and genetic algorithm
Li and Chen (2009)	New job arrivals and machine breakdown	A hybrid method based on artificial neural networks and genetic algorithms
Fattahi and Fallahi (2010)	New job arrivals, change in the processing times, adding new machine	Genetic algorithm
Adibi et al. (2010)	Random job arrivals, machine breakdown	Artificial neural networks and variable neighborhood search
Hao and Lin (2010)	Random job arrivals, machine breakdown	Interactive adaptive-weight evolutionary algorithm
Kapanoglu and Alikalfa (2011)	New job arrivals	Genetic algorithm
Hao and Gen (2011)	Random job arrivals, machine breakdown	Evolutionary algorithm
Qiu and Lau (2013)	New job arrivals, machine breakdown	A hybrid method based on artificial intelligence method of artificial immune systems (AIS) and priority dispatching rules
Nie et al. (2013)	New job arrivals	New heuristic method and gene expression programming
Zhang et al. (2013)	Random job arrivals, machine breakdown	Hybrid genetic algorithm and tabu search
Lu and Romanowski (2013)	New job arrivals	Dispatching rules
Sharma and Jain (2014)	Sequence-dependent setup times	Dispatching rules
Hosseinabadi et al. (2014)	Input of jobs	A new method called TIME_GELS
Nguyen et al. (2014)	Dynamic due-date assignment rule, dynamic total work content, dynamic processing and waiting	Four new multi-objective genetic programming-based hyper-heuristic (MO-GPHH) methods

characteristics of the dynamic job shop scheduling problem when machine breakdowns and new job arrivals occur and presented a hybrid approach involving neural networks and GA to solve this problem. Gao, Ding, and Zhang (2009) proposed a layered hybrid ant-colony and genetic algorithm for dynamic job shop scheduling problems to perform scheduling optimization which considers minimum completion time, minimum cost, maximum utilization rate, and minimum deviation degree as objectives. Adibi et al. (2010) proposed a hybrid method based on variable neighborhood search (VNS) and artificial neural network (ANN) for dynamic job shop scheduling that considers random job arrivals and machine breakdowns. Qiu and Lau (2013) proposed a hybrid method based on artificial immune systems (AIS) and priority dispatching rules for a dynamic online job shop scheduling problem assuming that new jobs continuously arrive at the job shop in a stochastic manner and the existence of unpredictable disturbances during the scheduling process. Zhang, Gao, and Li (2013) proposed a hybrid genetic algorithm and tabu search integrated with a simulator to solve the dynamic job shop scheduling problem with random job arrivals and machine breakdowns. Two performance measures, which contain the schedule efficiency and the schedule stability, are applied in the scheduling process. Nguyen, Zhang, Johnston, and Tan (2014) developed four new multi-objective genetic programming-based hyperheuristic (MO-GPHH) methods for automatic design of scheduling policies, including dispatching rules and due-date assignment rules in job shop environments. In addition to using three existing search strategies, nondominated sorting genetic algorithm II, strength Pareto evolutionary algorithm 2, and harmonic distance based multi-objective evolutionary algorithm, to develop new MO-GPHH methods, a new approach called diversified multi-objective cooperative evolution (DMOCC) is also proposed.

In Table 1, dynamic job shop scheduling applications in the literature are summarized with dynamic factors and solution methods. As there is a few research paper in the literature which address whole dynamic events such as new job arrivals, machine

breakdowns and the changes in the processing time, there is still some potential for developing fast and effective solutions to the problem. However, GA-based approaches, found in the literature, mainly consider DJSSs of unrealistic size. In real applications, on the other hand, the problem size is usually large and contains a lot of jobs to be allocated to the relevant machines. An obvious disadvantage of conventional GA approaches is that their performance rapidly deteriorates for problems with a larger number of machines and job operations, because an increased number of genes in a chromosome clearly reduces the efficiency of the GA operators. In order to overcome this weakness, this paper presents different hybrid GA based methodologies to obtain fast and efficient solutions in terms of solution quality and computational efficiency. First, we developed a new KK heuristic and it is combined with GA for dynamic job shop scheduling problems including new job arrival, machine breakdown, and changes in processing time. Moreover, well-known dispatching rules are integrated with GA to provide outstanding initial population. In addition, there is a lack of literature on detailed benchmark instances that compare different solution methodologies. Therefore, various benchmark problems including the number of jobs, the number of machines, and different dynamic events are generated based on Lawrence (1984) static job shop benchmark problem instances and presented in the paper to evaluate the performance of the proposed methodologies. Finally, we provide a numerical evaluation, which demonstrates the superiority of the hybrid GA approaches over the algorithms presented in the literature.

3. Dynamic job shop scheduling problem

The dynamic job shop scheduling (DJSS) problem is a combinatorial optimization problem and considers that n jobs are scheduled at the beginning of the schedule and a set of n' new jobs arrive after the start of the schedule is to be processed on m

machines. The DJSS problem subjects to the following assumptions (Zhang et al., 2013):

- Each machine can perform only one operation of any job at a time.
- An operation of a job can be performed by only one machine at a time.
- All machines are available at time 0.
- Once an operation has been processed on a machine, it must not be interrupted except due to machine breakdown. If an operation is interrupted by a machine breakdown, the remained processing time is equal to total processing time minus the completed processing time.
- An operation of a job cannot be performed until its preceding operations are completed.
- There is no flexible routing for each job.
- Operation processing time and the number of operable machines are known in advance. But there will be a change in the operation processing time and machines can breakdown.

The following notations are used for the problem formulation:
Definition of indices, sets, parameters and decision variables

j	old jobs ($j = 1, \dots, n$)
j'	new jobs ($j' = 1, 2, \dots, n'$)
i	machines ($i = 1, \dots, m$)
A	the set of all routing constraints $(i, j) \rightarrow (h, j)$
A'	the set of all routing constraints including new jobs $(i, j') \rightarrow (h, j')$
p_{ij}	the process time of the operation (i, j)
$p'_{ij'}$	the process time of the new operation (i, j')
rp	the start time of the rescheduling period
tm_i	the time that the machine i will be idled at the start of the rescheduling period
z_{ijk}	$=1$, if J_j precedes J_k on machine M_i ($=0$ otherwise)
$z'_{ij'k}$	$=1$, if $J'_{j'}$ precedes J'_k on machine M_i ($=0$ otherwise)
t_{ij}	$=1$, if J_j will be processed on machine M_i after rescheduling ($=0$ otherwise)
$t'_{ij'}$	$=1$, if $J'_{j'}$ will be processed on machine M_i after rescheduling ($=0$ otherwise)
c_{ij}	completion time of job j on machine i
$c'_{ij'}$	completion time of job j' on machine i
y_{ij}	the starting time of the operation (i, j)
$y'_{ij'}$	the starting time of the new operation (i, j')

Model formulation:

$$\text{Min } C_{\max} \quad (1)$$

s.t.

$$C_{\max} \geq c_{ij} \quad (i = 1, \dots, m, j = 1, \dots, n) \quad (2)$$

$$C_{\max} \geq c'_{ij'} \quad (i = 1, \dots, m, j' = 1, \dots, n') \quad (3)$$

$$c_{ij} = y_{ij} + p_{ij} \quad (i = 1, \dots, m, j = 1, \dots, n) \quad (4)$$

$$c'_{ij'} = y'_{ij'} + p'_{ij'} \quad (i = 1, \dots, m, j' = 1, \dots, n') \quad (5)$$

$$y_{hj} - y_{ij} \geq p_{ij} \quad \text{for all } (i, j) \rightarrow (h, j) \in A \quad (6)$$

$$y'_{hj'} - y'_{ij'} \geq p'_{ij'} \quad \text{for all } (i, j') \rightarrow (h, j') \in A' \quad (7)$$

$$Mz_{ijk} + (y_{ij} - y_{ik}) \geq p_{ik} \quad (i = 1, \dots, m, 1 \leq j \leq k \leq n) \quad (8)$$

$$Mz'_{ij'k} + (y'_{ij'} - y'_{ik}) \geq p'_{ik} \quad (i = 1, \dots, m, 1 \leq j' \leq k \leq n) \quad (9)$$

$$M(1 - z_{ijk}) + (y_{ik} - y_{ij}) \geq p_{ij} \quad (i = 1, \dots, m, 1 \leq j \leq k \leq n) \quad (10)$$

$$M(1 - z'_{ij'k}) + (y'_{ik} - y'_{ij'}) \geq p'_{ij'} \quad (i = 1, \dots, m, 1 \leq j' \leq k \leq n) \quad (11)$$

$$y_{ij} \geq (tm_i + rp)t_{ij} \quad (i = 1, \dots, m, j = 1, \dots, n) \quad (12)$$

$$y'_{ij'} \geq (tm_i + rp)t'_{ij'} \quad (i = 1, \dots, m, j' = 1, \dots, n') \quad (13)$$

$$y_{ij} \geq 0 \quad \text{for all } (i = 1, \dots, m, j = 1, \dots, n) \quad (14)$$

$$y'_{ij'} \geq 0 \quad \text{for all } (i = 1, \dots, m, j' = 1, \dots, n') \quad (15)$$

$$z_{ijk} = 0 \text{ or } 1 \quad (i = 1, \dots, m, j = 1, \dots, n) \quad (16)$$

$$z'_{ij'k} = 0 \text{ or } 1 \quad (i = 1, \dots, m, j' = 1, \dots, n') \quad (17)$$

$$t_{ij} = 0 \text{ or } 1 \quad (i = 1, \dots, m, j = 1, \dots, n) \quad (18)$$

$$t'_{ij'} = 0 \text{ or } 1 \quad (i = 1, \dots, m, j' = 1, \dots, n') \quad (19)$$

In this formulation, the objective function (1) minimizes the maximum completion time (makespan). Constraints (2) and (3) ensure that $C_{\max}$ is greater or equal than the completion time of job j and job j' on machine i . The completion time of each operation is calculated on constraints (4) and (5). Constraints (6) and (7) are precedence constraints. Each operation can be executed when its precedence operation was executed. Constraint sets (8)–(11) satisfy the requirement that only one job may be processed on a machine at any time. Because z_{ijk} is a 0–1 integer and adding M , a large enough positive number, has the effect of eliminating the associated constraint. Constraints (12) and (13) enforce operations, in the rescheduling period, to be started after the rescheduling time. Also each operation can be started when its related machine idled. The non-negativity restriction for y_{ij} , $y'_{ij'}$ and 0–1 restrictions for z_{ijk} , $z'_{ij'k}$ and t_{ij} , $t'_{ij'}$ are specified respectively in constraints (14)–(19).

Dynamic scheduling has been classified as completely reactive scheduling, predictive-reactive scheduling, and robust pro-active scheduling (Mehta & Uzsoy, 1999; Vieira, Hermann, & Lin, 2003). In this paper, predictive-reactive scheduling approach is considered for the dynamic job shop scheduling problem. The main issue in this approach is either to reschedule the facility each time a dynamic event like machine breakdown or new job arrival occurs (referred to as continuous rescheduling), or to reschedule periodically which is referred to as periodic rescheduling (Church & Uzsoy, 1992). In this paper, the continuous rescheduling approach, which provides better realized schedule performance, is used. In other words, rescheduling is performed each time a dynamic event such as a machine breakdown, new job arrival, or change in processing time occurs.

4. Proposed methods for dynamic job shop scheduling

The focus of this section is to develop hybrid GA methodologies to solve the dynamic job shop scheduling problem explained in the previous section. As there have been few research papers that propose GAs for the problems, there is still some potential for developing resourceful solutions. Therefore, we propose hybrid GA methodologies to obtain fast and efficient solutions especially for large size problems.

Table 2

Data of a job shop scheduling problem.

Jobs	Machine sequence	Processing times
1	J ₁₁ , J ₂₁ , J ₃₁ , J ₄₁	$p_{11} = 10, p_{21} = 8, p_{31} = 4, p_{41} = 2$
2	J ₂₂ , J ₁₂ , J ₄₂ , J ₃₂	$p_{22} = 8, p_{12} = 3, p_{42} = 5, p_{32} = 6$
3	J ₁₃ , J ₂₃ , J ₄₃ , J ₃₃	$p_{13} = 4, p_{23} = 7, p_{43} = 3, p_{33} = 2$

In this section, the proposed KK (Kundakci & Kulak) heuristic, which provides an initial population of Genetic Algorithms, is explained first. Then, brief information about well-known dispatching rules, which are also used to generate initial population, is presented. Finally, the structure of Genetic Algorithms and TS algorithm for solving the dynamic job shop scheduling problem and proposed GA-based methodologies are clarified in detail.

The steps of the KK approach can be summarized as follows:

Step1: Assign the first operations of the jobs to the relevant machines. If there is more than one operation to be initially assigned to a machine, assign the operation with the longer processing time. If the operations have equal processing times, assign one of them randomly.

Step 2: Assign the next operations of the jobs. If there is more than one operation to be assigned to a machine in the same order, assign the operation whose job predecessor was completed earliest. If the job predecessors of these operations were completed at the same time, assign one of them randomly.

Step 3: Repeat Step 2 until all operations have been assigned.

The proposed KK approach can be illustrated in the following example. The data of a 3×4 (job * machine) size job shop scheduling problem can be seen in Table 2.

According to the KK approach, the first operations of the jobs (J₁₁, J₂₂, J₁₃) are assigned to the relevant machines. As J₁₁ and J₁₃ must be assigned to the first machine in the first order, the operation J₁₁, which has a longer processing time than J₁₃, is assigned before J₁₃ to M₁. After completing assignment of the first operations of the jobs, the next step is to assign the second operations of the jobs (J₂₁, J₁₂, J₂₃) to the relevant machines. J₂₁ and J₂₃ must be assigned to M₂. As J₂₁'s job predecessor (J₁₁), was completed earlier than J₂₃'s job predecessor (J₁₃), J₂₁ is assigned to M₂ before J₂₃. The other operations of the jobs are assigned to the relevant machines by following step 2 of the KK approach and a Gantt chart of the job shop scheduling is developed as illustrated in Fig. 1.

SPT, LPT, SRPT and LRPT dispatching rules are explained below:

- SPT (Shortest Processing Time First): Highest priority is given to the waiting operation with the shortest operation time.
- LPT (Longest Processing Time First): Highest priority is given to the waiting operation with the longest operation time.

- SRPT (Shortest Remaining Processing Time First): Highest priority is given to the waiting operation associated with the job having the least total processing time remaining to be done.
- LRPT (Longest Remaining Processing Time First): Highest priority is given to the waiting operation associated with the job having the most amount of total processing time remaining to be done.

In order to improve the solution quality of GAs, a new KK heuristic is developed and integrated with GA. Well known dispatching rules are also combined with GA to generate efficient initial population for the dynamic job shop scheduling problem. In this way, GA-based methodologies, called GAKK, GASPT, GALPT, GASRPT, GALRPT and GAM, are proposed in this paper. The flow chart of the proposed methods can be seen in Fig. 2.

The GAKK algorithm is constructed by integrating the KK Heuristic with Genetic algorithm. Unlike typical GA, an initial solution is first generated using the KK heuristic + swap to generate 25% of the initial population. The rest of the population is randomly generated. In GASPT (Genetic Algorithm-Shortest Processing Time First), GALPT (Genetic Algorithm-Longest Processing Time First), GASRPT (Genetic Algorithm-Shortest Remaining Processing Time First) and GALRPT (Genetic Algorithm-Longest Remaining Processing Time First), 25% of the initial population is generated with related dispatching rule + swap and the rest is randomly generated. Lastly, in GAM (Genetic Algorithm-Mixed), 25% of the initial population is generated with a 5% mixture of KK, SPT, LPT, SRPT, LRPT + swap and the rest is randomly generated. The aim of generating 25% of the initial population with KK + swap or other dispatching rules + swap is to provide faster and more efficient solutions than conventional GA. After generating the initial population with the methods mentioned above, the steps in Fig. 2 are followed until the termination condition is satisfied. As the job scheduling problem considered in this paper is dynamic, the steps that are indicated in Fig. 2 are repeated each time a dynamic event (machine breakdown, new job arrival and change in the processing time) occurs.

4.1. Genetic algorithms

Genetic Algorithms (GA), were developed by John Holland and his collaborators in the 1960s and 1970s. GA is based on the mechanics of natural selection and natural genetics (Goldberg, 1989) and starts with a set of initial feasible solutions (generated randomly or by using some heuristic) called the initial population. Each individual in the population is called a chromosome that signifies a solution to the problem. Chromosomes consist of a series of elements called genes. A suitable encoding for the problem is required in GA and a fitness function that represents a measure

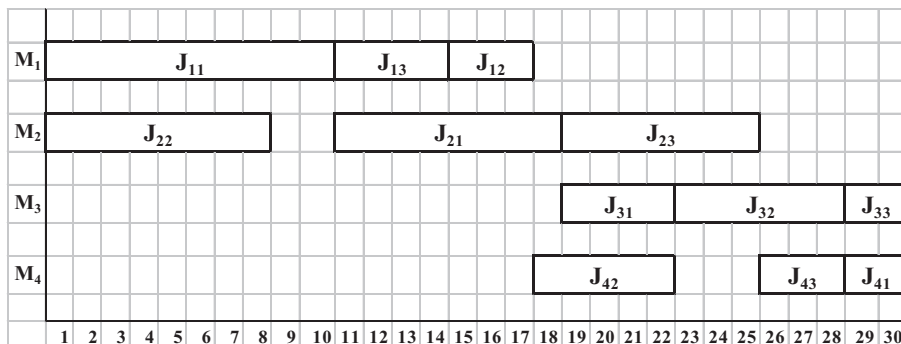


Fig. 1. Gantt chart of job shop scheduling using KK approach.

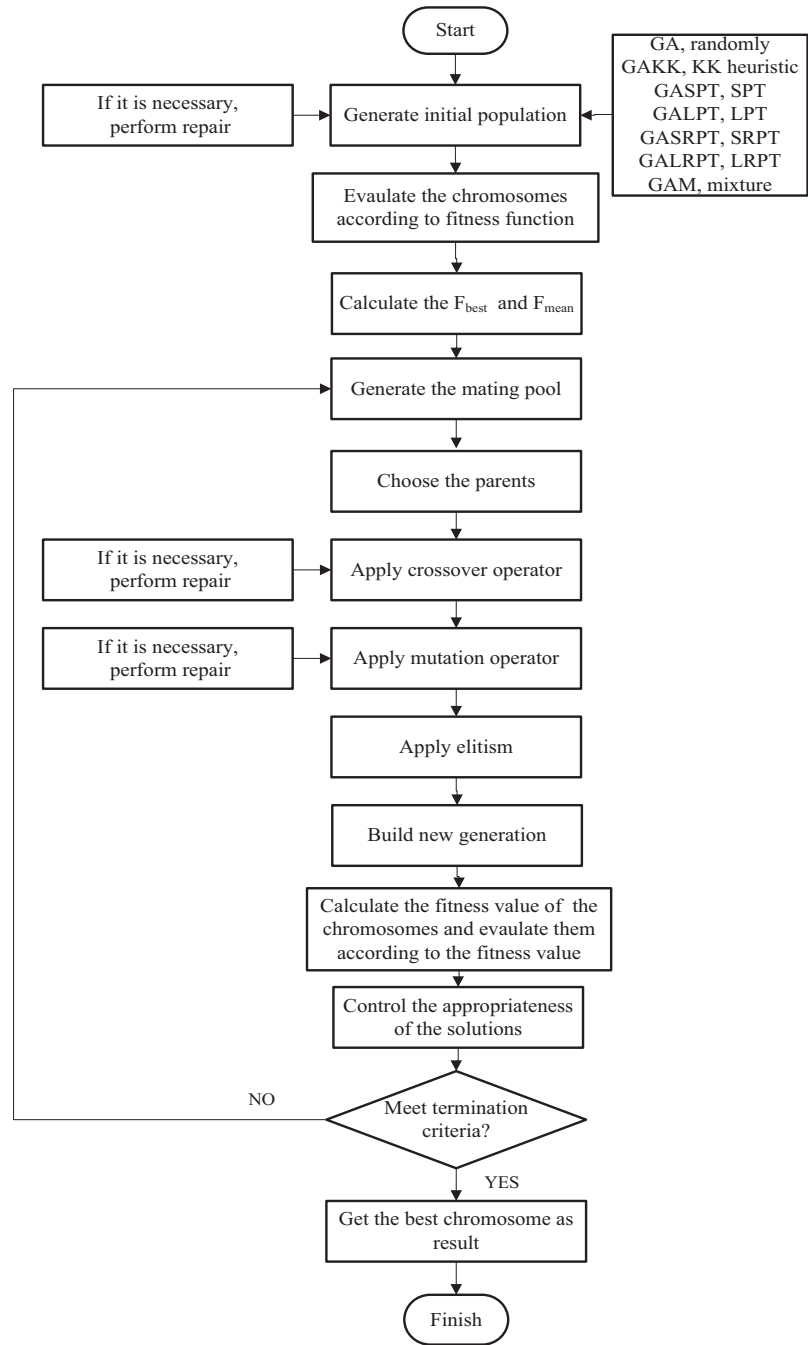


Fig. 2. Flow chart of GA.

of the quality of each chromosome. The reproduction tool selects the parents using a crossover operator to generate offspring chromosomes. These offspring chromosomes can randomly undergo a mutation operation by changing them locally and the portion of best chromosomes can be copied to the new population with elitism mechanism. After creating the new generation, the chromosomes are evaluated according to their fitness values. This process is repeated until the termination condition is satisfied.

4.1.1. Chromosome structure

The chromosome structure is composed of genes where the value of each gene identifies the operation sequence of the jobs in the related machines. An example of the chromosome structure for five jobs and four machines can be seen in Fig. 3.

4.1.2. Initial population

GA starts searching with a large randomly generated chromosome set referred to as the initial population. Generation of chromosomes with a very good fitness value is quite difficult due to randomness. A new KK heuristic + swap and well-known dispatching rules + swap are used to generate 25% of the GA initial population and the rest of the population is randomly generated. In the

M ₁					M ₂					M ₃					M ₄				
J ₃	J ₁	J ₄	J ₅	J ₂	J ₂	J ₄	J ₁	J ₅	J ₃	J ₁	J ₄	J ₂	J ₅	J ₃	J ₄	J ₃	J ₁	J ₂	J ₅

Fig. 3. Chromosome structure.

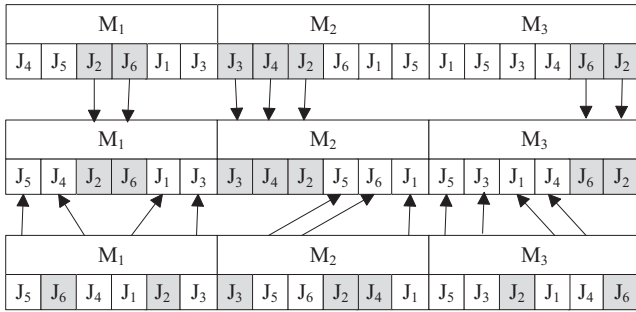


Fig. 4. An example of position based crossover operator.

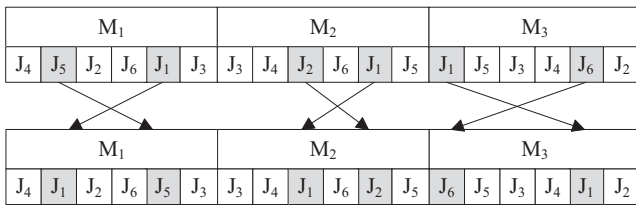


Fig. 5. An example of a swap mutation operator.

proposed methodology, half of the initial population delivering the better fitness values is selected as the GA population.

4.1.3. Fitness function

In each generation, all chromosomes in the population are evaluated by use of the fitness function. The chromosomes with better fitness values are included in the mating pool to form new offspring. Hence, selection of chromosomes with better fitness values enables the GA to reach solutions in a shorter period of time. The fitness function of the proposed GA is defined in Eq. (20). The chromosome offering the smallest fitness value is taken as the best current solution.

$$F_i = C_{max} = \text{Max}(C_1, C_2, \dots, C_n) \quad (20)$$

Here C_{max} indicates the maximum completion time (makespan).

4.1.4. Mating pool and selection

The mating pool is very important for GA performance by determining the parent chromosomes that are selected from the population. A mating pool is produced by selecting a number of best chromosomes from the current generation to generate new offspring chromosomes for the next generation. In this paper, a fitness scaling technique called σ -truncation (Goldberg, 1989) is used to

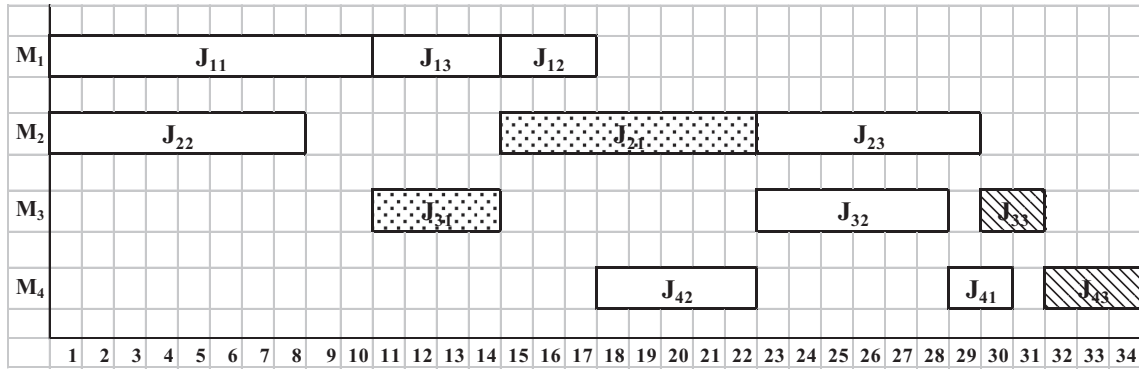


Fig. 6. Infeasible solution.

Table 3
Machine sequence of the jobs.

Jobs	Machine sequence	
	Feasible solution	Infeasible solution
1	J ₁₁ , J ₂₁ , J ₃₁ , J ₄₁	J ₁₁ , J ₃₁ , J ₂₁ , J ₄₁
2	J ₂₂ , J ₁₂ , J ₄₂ , J ₃₂	J ₂₂ , J ₁₂ , J ₄₂ , J ₃₂
3	J ₁₃ , J ₂₃ , J ₄₃ , J ₃₃	J ₁₃ , J ₂₃ , J ₃₃ , J ₄₃

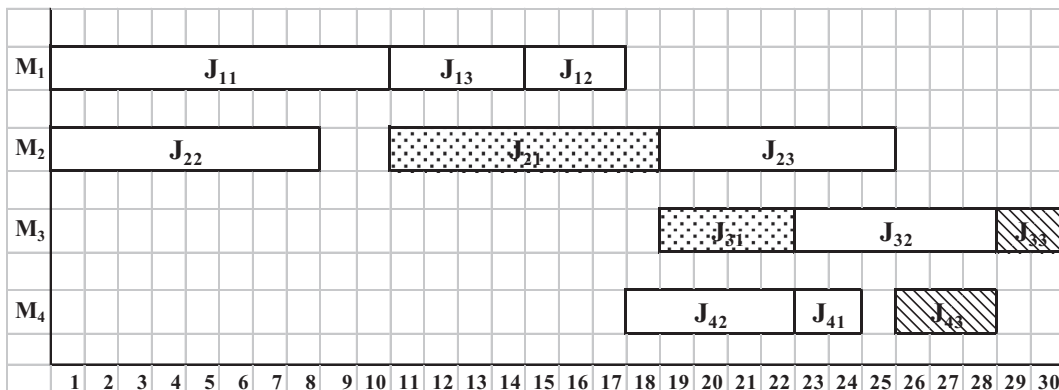


Fig. 7. Feasible solution.

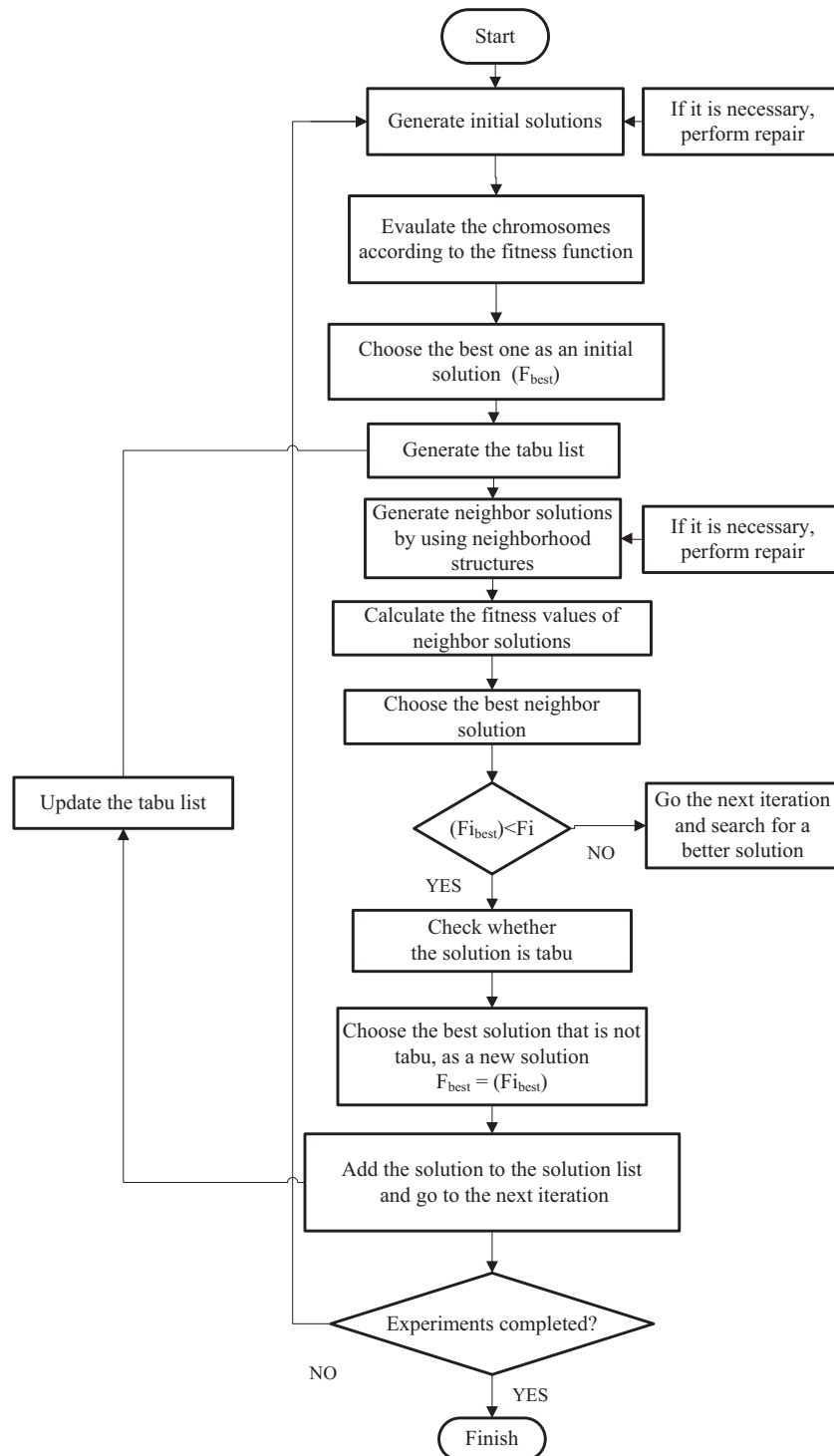


Fig. 8. Flow chart of TS.

Table 4
GA parameters.

Population size	Number of operations * 10
Crossover operator	Position based crossover (PBX)
Crossover ratio	0.90
Elitism ratio	0.10
Selection operator	Tournament
Mutation operator	Swap
Mutation ratio	0.10

Table 5
TS parameters.

Number of initial solutions	20
Type of tabu list	Dynamic
Tabu tenure	$0.9-1.1 \sqrt{n}$
Neighborhood search cycle	$10 * (\text{Number of jobs})$
Neighborhood structure	N7
Tabu search replication number	$10 * \text{Number of jobs} * \text{Number of machines}$

Table 6

Comparison of the GAM method with reference solutions known from the literature for static problem instances.

Problem	Size	SBI	SBII	TSAB	TSSB	GRASP	HGA-Param	HEA	MPSO	HIA	HGA	HBBO	GAM
FT06	6 * 6	55	55	55	55	55	55	55	55	55	55	55	55
FT10	10 * 10	1015	930	930	930	938	930	930	930	930	930	930	930
FT20	20 * 5	1290	1178	1165	1165	1169	1165	1169	1165	1165	1165	1165	1165
LA01	10 * 5	666	666	666	666	666	666	666	666	666	666	666	666
LA02	10 * 5	720	669	655	655	655	655	655	655	655	655	655	655
LA03	10 * 5	623	605	597	597	604	597	597	597	597	597	597	597
LA04	10 * 5	597	593	590	590	590	590	590	590	590	590	590	590
LA05	10 * 5	593	593	593	593	593	593	593	593	593	593	593	593
LA06	15 * 5	926	926	926	926	926	926	926	926	926	926	926	926
LA07	15 * 5	890	890	890	890	890	890	890	890	890	890	890	890
LA08	15 * 5	868	863	863	863	863	863	863	863	863	863	863	863
LA09	15 * 5	951	951	951	951	951	951	951	951	951	951	951	951
LA10	15 * 5	959	959	958	958	958	958	958	958	958	958	958	958
LA11	20 * 5	1222	1222	1222	1222	1222	1222	1222	1222	1222	1222	1222	1222
LA12	20 * 5	1039	1039	1039	1039	1039	1039	1039	1039	1039	1039	1039	1039
LA13	20 * 5	1150	1150	1150	1150	1150	1150	1150	1150	1150	1150	1150	1150
LA14	20 * 5	1292	1292	1292	1292	1292	1292	1292	1292	1292	1292	1292	1292
LA15	20 * 5	1207	1207	1207	1207	1207	1207	1207	1207	1207	1207	1207	1207
LA16	10 * 10	1021	978	945	945	946	945	945	945	945	945	945	945
LA17	10 * 10	796	787	784	784	784	784	784	784	784	784	784	784
LA18	10 * 10	891	859	848	848	848	848	848	848	848	848	848	848
LA19	10 * 10	875	860	842	842	842	842	–	842	842	844	842	842
LA20	10 * 10	924	914	902	902	907	907	–	902	902	907	902	902
LA21	15 * 10	1172	1084	1047	1046	1091	1046	1046	1046	1046	1046	1046	1046
LA22	15 * 10	1040	944	927	927	960	935	935	932	932	927	933	927
LA23	15 * 10	1061	1032	1032	1032	1032	1032	1032	1032	1032	1032	1032	1032
LA24	15 * 10	1000	976	939	938	978	953	–	941	950	953	935	935
LA25	15 * 10	1048	1017	977	979	1028	986	–	977	979	981	977	977
LA26	20 * 10	1304	1224	1218	1218	1271	1218	1218	1218	1218	1218	1218	1218
Average		937.76	912.52	903.79	903.79	912.24	905.00		904.00	904.38	904.62	90383	903.62
^a Deviation %		3.78	0.98	0.02	0.02	0.95	0.15		0.04	0.08	0.11	0.02	0.00

^a Deviation % = [(Obtained Average – Best Average)/Best Average].

select the chromosomes to be included in the mating pool. Tournament selection is used as selection operator. Tournament selection involves running several “tournaments” among a few individuals chosen randomly from the population. The winner of each tournament (the one with the best fitness value) is selected for crossover.

4.1.5. Crossover operator

There are several crossover operators in the literature regarding creation of new child chromosomes with the best fitness value. In this paper, position based crossover (PBX) is used as a crossover operator. A set of genes from the first parent is selected randomly with this operator. Then, an offspring chromosome is generated by copying these selected genes to the child directly. The other genes are copied from the second parent to the offspring in the same order from left to right. An example of PBX is given in Fig. 4.

4.1.6. Mutation operator

Mutation is a vital GA operator that allows the algorithm to enter different sections of the solution space during the search process. Thus, the acceptance of a local optimum as the best solution can be avoided with the help of this operator. Swap, inversion, left and right rotation operators are included in the proposed methodology. Another important part affecting the performance of the mutation operator is the definition of the mutation rate strategy. For GA applications, the use of a fixed mutation rate strategy for all generations is quite common. Nevertheless, an exponentially decreasing mutation rate strategy is also integrated into the proposed GA methodology. An example of a swap operator is given in Fig. 5.

4.1.7. Elitism

As an operational characteristic of GA, elitism provides a means for reducing genetic drift by ensuring that the best chromosome(s) can copy their traits to the next generation.

4.1.8. Repair strategy

In the initial solution, or after using genetic operators such as cross-over and mutation, infeasible chromosomes can be obtained. After generating infeasible chromosomes, a repair strategy is used to guarantee feasibility of all solutions. In a dynamic job shop when a predecessor of a job is processed after its job successor, infeasibility occurs. In other words, an infeasible solution exists when the precedence constraints are not satisfied. Therefore, the repair strategy is used to ensure feasible solutions.

In the repair strategy, the machine sequence of each job is considered. If the predecessor of a job is processed after its job successor, its place will be changed on the relevant machine. The repair strategy can be explained with an example using the data in Table 2. It is supposed that a random solution is obtained as in Fig. 6. However, this is an infeasible solution as the precedence constraints are not satisfied. According to the data in Table 3, in a feasible solution, the machine sequence of J_1 must be $J_{11}, J_{21}, J_{31}, J_{41}$. This means J_1 must be operated in M_1 first, then M_2, M_3 and M_4 respectively. But in the random solution presented in Fig. 6, the machine sequence of J_1 is $J_{11}, J_{31}, J_{21}, J_{41}$. Also in the feasible solution for J_3 , the machine sequence should be $J_{13}, J_{23}, J_{43}, J_{33}$, but is a random solution machine sequence of J_3 is $J_{13}, J_{23}, J_{33}, J_{43}$. So the repair strategy must be used to obtain a feasible solution.

According to the repair strategy, the placement of J_{21} and J_{31} must be changed to M_2 and M_3 respectively as seen in Fig. 7. In this way, the machine sequence of J_1 will be $J_{11}, J_{31}, J_{21}, J_{41}$. Similarly, the placement of J_{33} and J_{43} must be changed to M_3 and M_4 . After

Table 7
Comparison of the proposed methods.

Problem size (n × m)		Methods											
		GA			GAKK			GASPT			GALPT		
		C _{max}	CPU (sc.)	D ^a (%)	C _{max}	CPU	D (%)	C _{max}	CPU	D (%)	C _{max}	CPU	D (%)
5 × 5	57	2	5.88	53	53	1	3.92	53	2	3.92	51	2	0.00
6 × 5	581	12	4.31	557	557	5	0.00	572	9	2.69	573	11	2.87
8 × 5	757	10	8.30	757	757	4	8.30	699	9	0.00	779	12	11.44
10 × 5	624	46	0.00	624	624	15	0.00	624	38	0.00	624	39	0.00
10 × 6	710	173	4.11	682	682	42	0.00	738	104	8.21	707	95	3.67
15 × 5	1018	290	1.70	1001	1001	74	0.00	1032	226	3.10	1115	228	11.39
10 × 8	1317	384	28.24	1112	1112	295	8.28	1191	375	15.97	1269	324	23.56
10 × 9	1757	627	67.49	1049	1049	365	0.00	1635	435	55.86	1755	587	67.30
20 × 5	1900	348	39.60	1564	1564	339	14.92	1657	345	21.75	1898	341	39.46
10 × 10	3339	368	140.39	1410	1410	367	1.51	2310	364	66.31	2120	346	52.63
22 × 5	2807	559	92.52	1494	1494	354	2.47	2079	529	42.59	2046	435	40.33
12 × 10	2995	387	198.90	1046	1046	318	4.39	1640	352	60.68	1883	301	87.92
13 × 10	3227	337	217.62	1082	1082	329	6.50	2354	314	131.69	2186	328	115.16
20 × 7	3439	510	159.35	1340	1340	349	1.06	2927	459	120.74	2388	504	80.09
15 × 10	4541	490	254.77	1324	1324	448	3.44	3242	415	153.28	3176	472	148.13

^a Deviation = [(Obtained Value – Best Value)/Best Value].

using the repair strategy, a feasible solution like in Fig. 7 is obtained. In this solution, precedence constraints are satisfied.

4.1.9. Termination condition

Computational time and convergence of the fitness value are considered simultaneously for terminating the GA iterations. The search process is ended, if both the average and the best fitness values of the current population are equal to the corresponding values of the next generation for a given number of successive iterations (Kulak, Yilmaz, & Günther, 2007). Additionally, a CPU time limit is defined in order to restrict the computational effort. The algorithm is repeated until the termination condition is satisfied.

4.2. Tabu search algorithm

Tabu Search (TS), which was developed by Glover (1986), is a general framework for a variety of iterative local search strategies for discrete optimization. TS uses the concept of memory by controlling the algorithm's execution via a list of forbidden moves. This allows the TS algorithm to intensify or diversify its search of a given problem's solution space in an effort to avoid entrapment in local optimal (Gendreau & Potvin, 2010). TS algorithm starts to search with one initial solution and generally this solution is determined randomly. But in this paper, a set of initial solutions is determined and then these solutions are evaluated according to their fitness values and the best one is selected as an initial solution (Kulak, Sahin, & Taner, 2012). In this way, the TS algorithm starts with a better solution than a random one. The flow chart of the proposed TS method is given in Fig. 8. In this paper, a dynamic job shop scheduling problem is considered and the steps that are indicated in Fig. 8 are repeated every time a dynamic event (machine breakdown, new job arrival and change in the operation time) occurs.

4.2.1. Solution structure

The definition of the solution structure is the most difficult part of TS application. The structure is composed of segments where the value of each segment identifies the operation sequence of the jobs on the relevant machines.

4.2.2. Fitness function

The fitness function of the TS is the maximum completion time (makespan). The best solution is regarded as a solution structure that minimizes the makespan over all iterations. The fitness value obtained in the final iteration represents the best solution to the problem.

4.2.3. Neighborhood structure

In order to generate neighbor solutions, various neighborhood search methods can be used in TS. TS uses a neighborhood search procedure to iteratively move from a solution X to a solution X' in the neighborhood of X. There are different neighborhood structures proposed by different authors. These neighborhood structures can be given as:

- N1 (Van Laarhoven, Aarts, & Lenstra, 1992)
- N2 (Barnes & Chambers, 1995)
- N3 and N4 (Dell'Amico & Trubian, 1993)
- N5 (Nowicki & Smutnicki, 1996)
- N6 (Balas & Vazacopoulos, 1998)

In this paper, a new neighborhood structure has been developed to be referred to as N7. Before explaining the N7 neighborhood structure, some concepts must be explained. A *critical path* is the longest route from start to end in a Gantt chart and its length represents the makespan C_{max} . Any operation on the critical path is

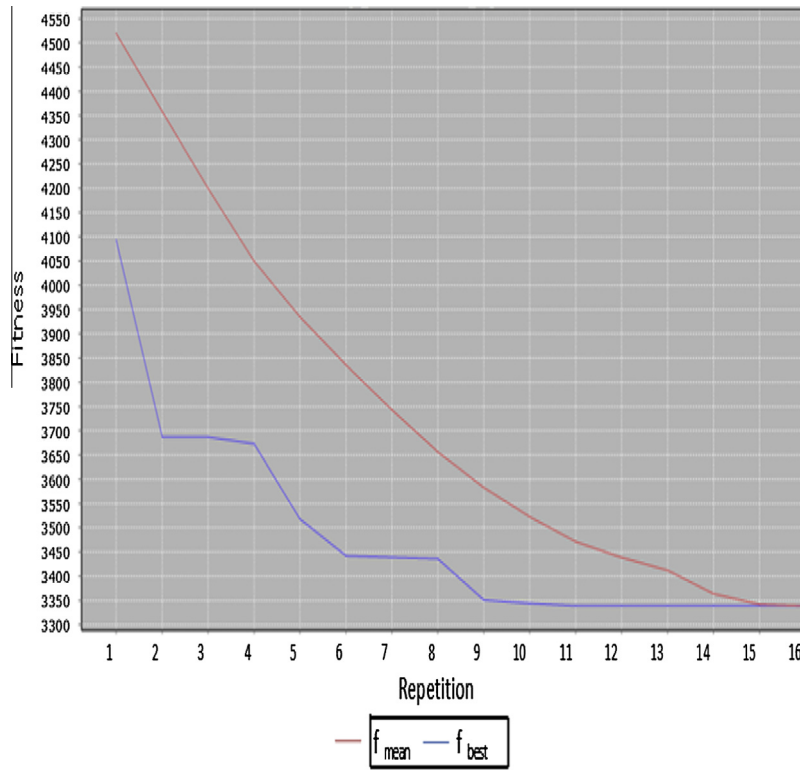


Fig. 9. Solution graph of 10×10 DJSS problem with GA.

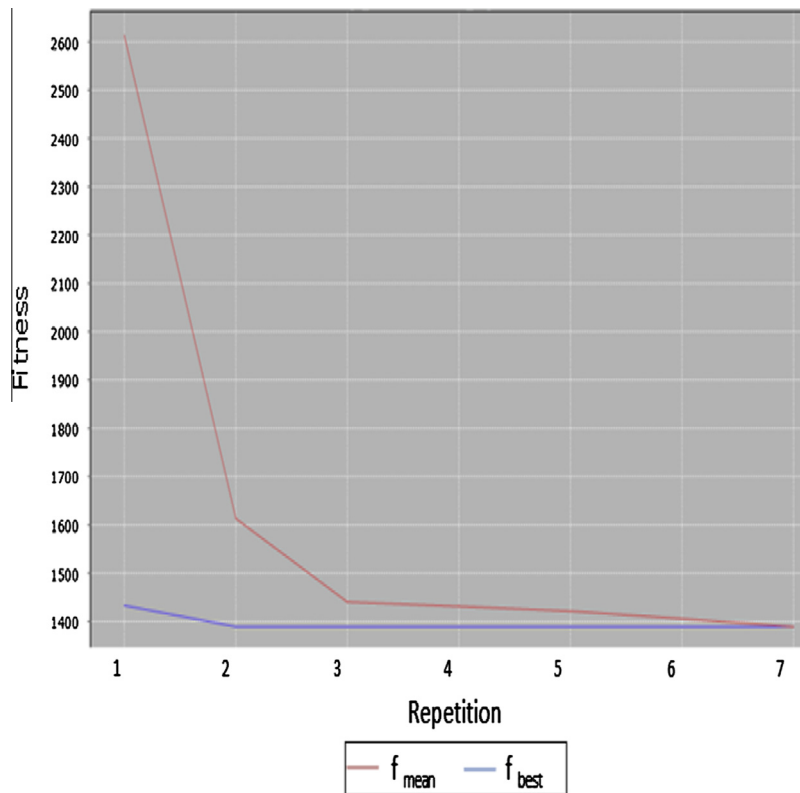


Fig. 10. Solution graph of 10×10 DJSS problem with GAM.

called a *critical operation*. A *block* is a maximal sequence of adjacent critical operations that processed on the same machine. Any operation, u , has two immediate predecessors and successors. Its job

predecessor and successor are denoted by $JP(u)$ and $JS(u)$ and its machine predecessor and successor are denoted by $MP(u)$ and $MS(u)$ (Zhang, Li, Guan, & Rao, 2007).

N7 neighborhood structure: First, *critical paths* are determined. Then, blocks are determined and examined. If the job predecessor JP(u) of the first job of a block is also on the critical block, the first two jobs of this block change places. Also, the first job of the block is changes places with its machine predecessors MP(u) and the last job of the block changes places with its machine successor MS(u).

4.2.4. Tabu restrictions

Tabu restrictions are essential tool that prevents moving back to previously investigated solutions. Therefore, once a move is executed, the move is classified as tabu. In general, there are at least three possible approaches namely fixed, dynamic or random. In this paper, the dynamic approach is selected based on our empirical tests which indicated that it was superior to other approaches. We also investigate the performance of the TS control parameters in Section 5 for all test instances.

4.2.5. Aspiration criteria

The aspiration criterion determines when a move can be executed even if it is a tabu restriction. In this paper, a commonly used aspiration criteria that consists in selecting a tabu move if it generates a solution being better than the best found solution is considered.

4.2.6. Stopping criteria

In this paper, the search process will be terminated if the CPU time limit is exceeded, or the number of iterations reaches the maximum allowable number.

5. Numerical investigations

In this section, the performance of the proposed methods is evaluated by considering the results of conducted experiments. First of all, the performance of the proposed GAM method is tested for the static job shop scheduling problem using the benchmark problem instances ft06, ft10, ft20 from Fisher and Thompson (1963) and LA01–LA26 from Lawrence (1984). This problem set includes 29 benchmark instances in which machine x job numbers vary between 36 and 200. Then the performance of the proposed methods are investigated for the dynamic job shop scheduling problem using new-generated benchmark instances. As there is a lack of literature on detailed benchmark instances that compare different solution methodologies, various benchmark problems including the number of jobs, the number of machines, and different dynamic events are generated based on Lawrence (1984) static job shop benchmark problem instances. This problem set is categorized as small, medium and large size problems and includes 15 benchmark instances.

5.1. Definition of best parameters used in GA and TS

In order to determine the best GA and TS parameters for our application, a number of preliminary tests were carried out. A test instance with 6 jobs and 6 machines is generated randomly, then experiments for each parameter set are conducted 5 times, and the results are evaluated with ANOVA testing. GA parameters determined after 7680 experiments are shown in Table 4. Furthermore, the best parameters used in a TS algorithm are shown in

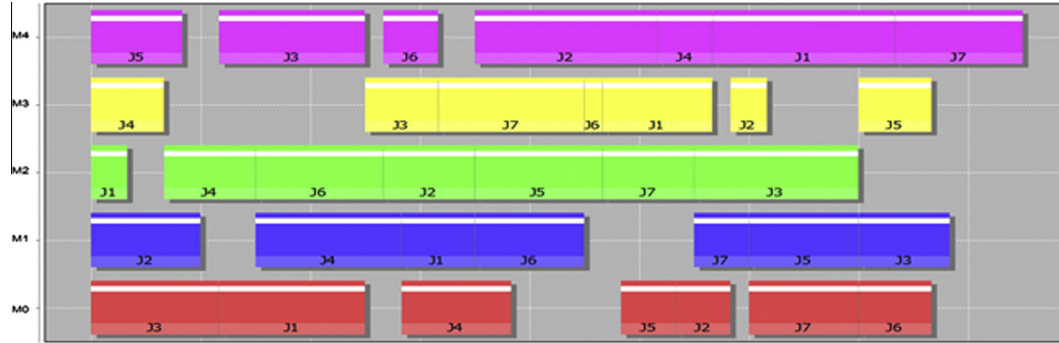


Fig. A.1. Gantt chart of the 5 × 5 dynamic job scheduling problem with GAM.

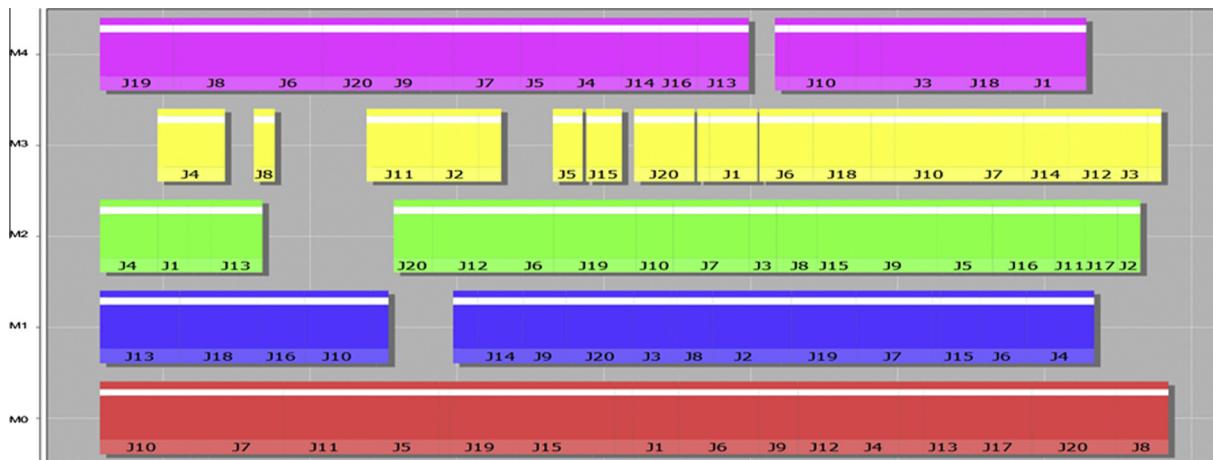


Fig. A.2. Gantt chart of the LA11 static job scheduling problem with GAM.

Table A.1Data of 5×5 dynamic job scheduling problem.

Machine sequences and processing times													
Jobs	Machine sequences and processing times												
J1	M2	2	M0	8	M1	4	M3	6	M4	7			
J2	M1	6	M2	5	M4	4	M0	3	M3	2			
J3	M0	7	M4	8	M3	4	M2	9	M1	3			
J4	M3	4	M2	5	M1	5	M0	4	M4	3			
J5	M4	5	M2	7	M0	3	M1	6	M3	4			
Dynamic events													
Dynamic events	Occurrence time	Event type ^a	Information about dynamic events										
D1	12	0	M1	3									
D2	23	0	M4	6									
D3	5	1	J6	M2	7	M4	3	M1	6	M3	1	M0	4
D4	18	1	J7	M3	8	M2	5	M1	3	M0	6	M4	7
D5	0	2	J4	M0	6								
D6	0	2	J1	M4	10								
D7	0	2	J3	M1	5								

^a 0: Machine breakdown. 1: New job arrival. 2: Change in the process time.**Table A.2**Solution of 5×5 dynamic job scheduling problem with GAM.

Machines	M0	Job Sequence	J3	J1	J4	J5	J2	J7	J6
		Start Time	0	7	17	29	32	36	42
		Finish Time	7	15	23	32	35	42	46
	M1	Job Sequence	J2	J4	J1	J6	J7	J5	J3
		Start Time	0	9	17	21	33	36	42
		Finish Time	6	17	21	27	36	42	47
	M2	Job Sequence	J1	J4	J6	J2	J5	J7	J3
		Start Time	0	4	9	16	21	28	33
		Finish Time	2	9	16	21	28	33	42
	M3	Job Sequence	J4	J3	J7	J6	J1	J2	J5
		Start Time	0	15	19	27	28	35	42
		Finish Time	4	19	27	28	34	37	46
	M4	Job Sequence	J5	J3	J6	J2	J4	J1	J7
		Start Time	0	7	16	21	31	34	44
		Finish Time	5	15	19	31	34	44	51

Table A.3Data of 6×5 dynamic job scheduling problem.

Machine sequences and processing times													
Jobs	Machine sequences and processing times												
J1	M1	21	M2	34	M0	53	M4	21	M3	55			
J2	M4	55	M2	31	M0	12	M1	42	M3	39			
J3	M3	34	M2	64	M1	19	M4	92	M0	83			
J4	M1	87	M3	69	M2	87	M0	93	M4	60			
J5	M2	98	M0	44	M1	49	M4	96	M3	17			
J6	M2	35	M3	76	M4	61	M0	95	M1	7			
Dynamic events													
Dynamic Events	Occurrence time	Event type	Information about dynamic events										
D1	15	0	M3	9									
D2	72	0	M1	5									
D3	38	1	J7	M3	22	M1	32	M0	10	M2	25	M4	54
D4	46	1	J8	M2	56	M4	27	M3	32	M1	49	M0	9
D5	110	1	J9	M1	10	M0	23	M2	47	M3	27	M4	8
D6	0	2	J4	M0	48								
D7	0	2	J3	M1	23								

Table 5. The parameters defined in these tables will be used throughout our numerical experiments.

5.2. Comparison of the proposed methods

The performance of the proposed GAM method is tested using the benchmark problem set ft06, ft10, ft20 from Fisher and

Thompson (1963) and LA01–LA26 from Lawrence (1984). This problem set includes 29 problem instances in which machine $\times$ job numbers vary between 36 and 200. The proposed GAM method is compared in Table 6 with reference solutions known from the literature for benchmark problem instances, namely SBI by Joseph, Egon, and Daniel (1988), SBII by Joseph et al. (1988), TSAB by Nowicki and Smutnicki (1996), TSSB by Pezzella and Merelli (2000), GRASP by

Table A.4

Data of 8×5 dynamic job scheduling problem.

Machine sequences and processing times												
Jobs	Machine sequences and processing times											
J1	M0	12	M2	94	M3	92	M4	91	M1	57		
J2	M1	19	M3	11	M4	66	M2	21	M0	87		
J3	M1	14	M0	75	M3	13	M4	16	M2	20		
J4	M2	95	M4	66	M0	47	M3	77	M1	75		
J5	M1	45	M3	26	M4	89	M0	15	M2	34		
J6	M3	77	M2	20	M0	76	M4	88	M1	53		
J7	M2	74	M1	88	M0	52	M3	27	M4	91		
J8	M1	88	M3	69	M0	62	M4	98	M2	52		
Dynamic events												
Dynamic events	Occurrence time	Event type	Information about dynamic events									
D1	21	0	M4	16								
D2	245	0	M2	9								
D3	170	1	J9	M2	47	M3	36	M1	12	M0	10	M4 57
D4	0	2	J7	M0	55							
D5	0	2	J6	M2	24							

Table A.5

Data of 10×5 dynamic job scheduling problem.

Machine sequences and processing times												
Jobs	Machine sequences and processing times											
J1	M1	72	M0	87	M4	95	M2	66	M3	60		
J2	M4	15	M3	35	M0	48	M2	39	M1	54		
J3	M1	46	M3	20	M2	21	M0	97	M4	55		
J4	M0	59	M3	19	M4	46	M1	34	M2	37		
J5	M4	23	M2	73	M3	25	M1	24	M0	28		
J6	M3	28	M0	45	M4	45	M1	78	M2	83		
J7	M0	53	M3	71	M1	37	M4	29	M2	12		
J8	M4	12	M2	87	M3	33	M1	55	M0	38		
J9	M2	49	M3	83	M1	40	M0	48	M4	27		
J10	M2	65	M3	17	M0	90	M4	27	M1	23		
Dynamic events												
Dynamic events	Occurrence time	Event type	Information about dynamic events									
D1	26	0	M0	8								
D2	178	0	M2	12								
D3	269	0	M3	5								
D4	45	1	J11	M3	48	M1	17	M0	23	M4	59	M2 36
D5	0	2	J7	M1	46							
D6	0	2	J9	M4	13							
D7	0	2	J1	M4	100							

Table A.6

Data of 10×6 dynamic job scheduling problem.

Machine sequences and processing times												
Jobs	Machine sequences and processing times											
J1	M3	60	M4	48	M5	95	M0	87	M1	72	M2	66
J2	M0	97	M5	55	M2	21	M3	20	M4	59	M1	46
J3	M4	45	M2	73	M1	24	M0	28	M3	25	M5	23
J4	M0	53	M2	12	M1	37	M3	71	M5	29	M4	38
J5	M4	90	M2	49	M5	7	M0	48	M3	83	M1	40
J6	M3	85	M4	25	M2	84	M1	66	M0	62	M5	19
J7	M5	88	M4	14	M0	41	M1	73	M2	53	M3	80
J8	M1	78	M5	64	M4	63	M3	84	M0	84	M2	41
J9	M1	11	M0	64	M2	17	M4	85	M5	73	M3	10
J10	M3	93	M2	95	M1	65	M0	59	M5	46	M4	60
Dynamic events												
Dynamic events	Occurrence time	Event type	Information about dynamic events									
D1	137	0	M5	6								
D2	286	0	M2	14								
D3	472	0	M0	5								
D4	45	1	J11	M1	17	M3	26	M5	11	M2	9	M4 55 M0 48
D5	0	2	J10	M0	65							
D6	0	2	J3	M5	27							
D7	0	2	J7	M4	18							

Table A.7Data of 15×5 dynamic job scheduling problem.

Machine sequences and processing times													
Jobs	Machine sequences and processing times												
J1	M1	21	M2	34	M4	95	M0	53	M3	55			
J2	M3	52	M4	16	M1	71	M2	26	M0	21			
J3	M2	31	M0	12	M1	42	M3	39	M4	98			
J4	M3	77	M1	77	M4	79	M0	55	M2	66			
J5	M4	37	M3	34	M2	64	M1	19	M0	83			
J6	M2	43	M1	54	M0	92	M3	62	M4	79			
J7	M0	93	M3	69	M1	87	M4	77	M2	87			
J8	M0	60	M1	41	M2	38	M4	83	M3	24			
J9	M2	98	M3	17	M4	25	M0	44	M1	49			
J10	M0	96	M4	77	M3	79	M1	75	M2	43			
J11	M4	28	M2	35	M0	95	M3	76	M1	7			
J12	M0	61	M4	10	M2	95	M1	9	M3	35			
J13	M4	59	M3	16	M1	91	M2	59	M0	46			
J14	M4	43	M1	52	M0	28	M2	27	M3	50			
J15	M0	87	M1	45	M2	39	M4	9	M3	41			
Dynamic events													
Dynamic events	Occurrence time	Event type	Information about dynamic events										
D1	148	0	M0	8									
D2	450	0	M4	17									
D3	200	1	J16	M1	70	M0	67	M2	88	M3	24	M4	16
D4	0	2	J5	M3	46								

Table A.8Data of 10×8 dynamic job scheduling problem.

Machine sequences and processing times																			
Jobs	Machine sequences and processing times																		
J1	M0	96	M1	69	M2	25	M3	5	M4	55	M5	15	M6	88	M7	11			
J2	M0	11	M1	48	M2	67	M3	38	M4	18	M7	24	M6	62	M5	92			
J3	M2	67	M1	63	M0	93	M4	85	M3	25	M5	72	M6	51	M7	81			
J4	M2	30	M1	35	M0	27	M4	82	M3	44	M7	92	M6	25	M5	49			
J5	M1	53	M0	83	M4	73	M3	26	M2	77	M6	33	M5	92	M7	38			
J6	M1	20	M0	44	M4	81	M3	88	M2	66	M6	70	M5	91	M7	96			
J7	M1	21	M2	93	M4	22	M0	56	M3	34	M6	40	M7	53	M5	29			
J8	M1	32	M2	63	M4	36	M0	26	M3	17	M5	85	M7	15	M6	82			
J9	M0	73	M2	46	M3	89	M4	24	M1	99	M6	92	M7	7	M5	19			
J10	M0	52	M2	20	M3	70	M4	98	M1	23	M5	15	M7	81	M6	81			
Dynamic events																			
Dynamic events	Occurrence time	Event type	Information about dynamic events																
D1	37	0	M2	7															
D2	148	0	M6	11															
D3	66	1	J11	M2	5	M7	21	M6	64	M0	47	M1	24	M3	88	M4	9	M5	17
D4	0	2	J1	M3	9														
D5	0	2	J6	M2	73														
D6	0	2	J7	M1	26														

Binato, Hery, Loewenstern, and Resende (2002), HGA-Param by Gonçalves, Magalhães Mendes, and Mauricio (2005), HEA by Ge, Du, and Qian (2007), HIA by Ge, Sun, Liang, and Qian (2008), MPSP by Lin, Horng, Kao, Chen, and Run (2010), HGA by Qing-dao-er-ji and Wang (2012) and HBBO by Wang and Duan (2014).

The proposed GAM method was run 5 times with the same problem instance and best solution for the proposed method is given in the right part of Table 6. Overall best solutions of each problem instance are highlighted in bold. Comparing GAM with the reference solutions from literature it can be seen that GAM method reaches the best known solution for all problem instances. For instance, the solution and Gantt chart for LA11 benchmark problem instance is given in Appendix A in Table A.17 and Fig. A.2 respectively. From the comparison of the individual heuristics the following conclusions can be drawn. GAM shows the best

average solution (903.62) for all problem instances compared to 11 benchmark heuristics. Moreover, GAM shows the smallest gap when compared with the average of best known solutions for the range of problem instances over all solution approaches. On average over 29 problem instances, GAM performs 0.02% better than the best counterparts from literature. In comparison to individual heuristics from literature, the proposed GAM approach yields superior solutions on average. We note here that for most of the published solution approaches computational times are not provided. The only computational time of HBBO approach by Wang and Duan (2014) for the FT06, is given as 0.54 s. The computational time of the proposed GAM method for the same instance is obtained as 0.16 s. The general conclusion that can be drawn from Tables 6 is that the GAM method produces adequate solutions in reasonable time for the all benchmark problems.

Table A.9

Data of 10×9 dynamic job scheduling problem.

Machine sequences and processing times																			
Jobs	Machine sequences and processing times																		
J1	M7	62	M8	24	M5	25	M3	84	M4	47	M6	38	M2	82	M0	93	M1	66	
J2	M5	47	M2	97	M8	92	M1	93	M4	29	M7	56	M3	80	M0	78	M6	67	
J3	M1	45	M7	46	M6	22	M2	26	M0	69	M4	40	M3	33	M8	75	M5	96	
J4	M4	85	M8	76	M5	68	M3	36	M6	75	M2	56	M1	35	M0	77	M7	85	
J5	M8	60	M7	25	M3	63	M4	81	M0	52	M1	30	M5	98	M6	54	M2	86	
J6	M3	87	M5	51	M2	95	M4	65	M1	86	M6	22	M8	58	M0	80	M7	65	
J7	M5	81	M2	53	M7	57	M6	71	M0	43	M4	26	M8	54	M3	58	M1	69	
J8	M4	20	M6	86	M5	21	M8	79	M2	34	M0	27	M1	81	M7	30	M3	46	
J9	M6	66	M5	98	M8	86	M7	66	M0	56	M3	82	M1	95	M4	47	M2	78	
J10	M0	30	M3	50	M7	34	M2	58	M1	77	M5	34	M8	84	M4	40	M6	44	
Dynamic events																			
Dynamic events		Occurrence time		Event type		Information about dynamic events													
D1		170		0		M6 24													
D2		97		0		M4 6													
D3		259		0		M7 15													
D4		270		1		J11 M1 76 M4 48 M3 29 M6 57 M2 73 M7 69 M0 68 M5 53 M8 37													
D5		320		1		J12 M3 48 M0 76 M1 43 M2 74 M8 46 M5 38 M4 57 M7 27 M6 49													
D6		0		2		J4 M5 76													
D7		0		2		J7 M6 84													
D8		0		2		J9 M0 39													

Table A.10

Data of 20×5 dynamic job scheduling problem.

Machine sequences and processing times													
Jobs	Machine sequences and processing times												
J1	M0	29	M1	9	M2	49	M3	62	M4	44			
J2	M0	43	M1	75	M3	69	M2	46	M4	72			
J3	M1	91	M0	39	M2	90	M4	12	M3	45			
J4	M1	81	M0	71	M4	9	M2	85	M3	22			
J5	M2	14	M1	22	M0	26	M3	21	M4	72			
J6	M2	84	M1	52	M4	48	M0	47	M3	6			
J7	M1	46	M0	61	M2	32	M3	32	M4	30			
J8	M2	31	M1	46	M0	32	M3	19	M4	36			
J9	M0	76	M3	76	M2	85	M1	40	M4	26			
J10	M1	85	M2	61	M0	64	M3	47	M4	90			
J11	M1	78	M3	36	M0	11	M4	56	M2	21			
J12	M2	90	M0	11	M1	28	M3	46	M4	30			
J13	M0	85	M2	74	M1	10	M3	89	M4	33			
J14	M2	95	M0	99	M1	52	M3	98	M4	43			
J15	M0	6	M1	61	M4	69	M2	49	M3	53			
J16	M1	2	M0	95	M3	72	M4	65	M2	25			
J17	M0	37	M2	13	M1	21	M3	89	M4	55			
J18	M0	86	M1	74	M4	88	M2	48	M3	79			
J19	M1	69	M2	51	M0	11	M3	89	M4	74			
J20	M0	13	M1	7	M2	76	M3	52	M4	45			
Dynamic events													
Dynamic events	Occurrence time	Event type	Information about dynamic events										
D1	163	0	M1	10									
D2	56	1	J21	M3	25	M1	41	M0	84	M2	28	M4	72
D3	0	2	J4	M1	88								
D4	0	2	J10	M0	67								
D5	0	2	J17	M4	58								

The evaluation of solution methods for dynamic scheduling problems raises the need for a systematic generation of benchmark instances. Therefore, dynamic job shop problem instances are generated based on [Lawrence \(1984\)](#) static job shop benchmark problem instances. Each problem takes into account dynamic events such as machine breakdown, new job arrival and change in the processing time (see [Appendix A](#)). A set of computational experiments are generated based on using the generation procedure for dynamic job shop scheduling problems in different sizes (5×5 ,

6 × 5, 8 × 5, 10 × 5, 10 × 6, 15 × 5, 10 × 8, 10 × 9, 20 × 5, 10 × 10, 22 × 5, 12 × 10, 13 × 10, 20 × 7, 15 × 10). Problems which have less than 60 operations are categorized as small size problems (5 × 5, 6 × 5, 8 × 5, 10 × 5). Problems which have between 60 and 100 operations are categorized as medium size problems (10 × 6, 15 × 5, 10 × 8, 10 × 9) and problems which have 100 or more operations are categorized as large size problems (10 × 10, 22 × 5, 12 × 10, 13 × 10, 20 × 7, 15 × 10). The proposed methodology is coded in JAVA using a Microsoft Windows XP

Table A.11Data of 10×10 dynamic job scheduling problem.

Machine sequences and processing times																					
Jobs	Machine sequences and processing times																				
J1	M4	88	M8	68	M6	94	M5	99	M1	67	M2	89	M9	77	M7	99	M0	86	M3	92	
J2	M5	72	M3	50	M6	69	M4	75	M2	94	M8	66	M0	92	M1	82	M7	94	M9	63	
J3	M9	83	M8	61	M0	83	M1	65	M6	64	M5	85	M7	78	M4	85	M2	55	M3	77	
J4	M7	94	M2	68	M1	61	M4	99	M3	54	M6	75	M5	66	M0	76	M9	63	M8	67	
J5	M3	69	M4	88	M9	82	M8	95	M0	99	M2	67	M6	95	M5	68	M7	67	M1	86	
J6	M1	99	M4	81	M5	64	M6	66	M8	80	M2	80	M7	69	M9	62	M3	79	M0	88	
J7	M7	50	M1	86	M4	97	M3	96	M0	95	M8	97	M2	66	M5	99	M6	52	M9	71	
J8	M4	98	M6	73	M3	82	M2	51	M1	71	M5	94	M7	85	M0	62	M8	95	M9	79	
J9	M0	94	M6	71	M3	81	M7	85	M1	66	M2	90	M4	76	M5	58	M8	93	M9	97	
J10	M3	50	M0	59	M1	82	M8	67	M7	56	M9	96	M6	58	M4	81	M5	59	M2	96	
Dynamic events																					
Dynamic events	Occurrence time		Event type		Information about dynamic events																
D1	72		0		M3 11																
D2	65		0		M1 10																
D3	95		1		J11 M2 43 M5 54 M1 24 M3 15 M6 48 M4 75 M7 85 M0 68 M9 74 M8 37																
D4	15		2		J2 M4 93																
D5	20		2		J10 M0 68																

Table A.12Data of 22×5 dynamic job scheduling problem.

Machine sequences and processing times													
Jobs	Machine sequences and processing times												
J1	M3	60	M0	87	M1	72	M4	95	M2	66			
J2	M1	54	M0	48	M2	39	M3	35	M4	5			
J3	M3	20	M1	46	M0	97	M2	21	M4	55			
J4	M2	37	M0	59	M3	19	M1	34	M4	46			
J5	M2	73	M3	25	M1	24	M0	28	M4	23			
J6	M1	78	M3	28	M2	83	M0	45	M4	5			
J7	M3	71	M1	37	M2	12	M4	29	M0	53			
J8	M4	12	M3	33	M1	55	M2	87	M0	38			
J9	M0	48	M1	40	M2	49	M3	83	M4	7			
J10	M0	90	M4	27	M2	65	M3	17	M1	23			
J11	M0	62	M3	85	M1	66	M2	84	M4	19			
J12	M3	59	M2	46	M4	13	M1	64	M0	25			
J13	M2	53	M1	73	M3	80	M4	88	M0	41			
J14	M2	57	M4	47	M0	14	M1	67	M3	74			
J15	M2	41	M4	64	M3	84	M1	78	M0	84			
J16	M4	52	M3	28	M2	26	M0	63	M1	46			
J17	M1	11	M0	64	M3	10	M4	73	M2	17			
J18	M4	38	M3	95	M0	85	M1	97	M2	67			
J19	M3	93	M1	65	M2	95	M0	59	M4	46			
J20	M0	60	M1	85	M2	43	M4	85	M3	32			
J21	M4	9	M2	29	M1	51	M3	26	M0	37			
J22	M3	77	M1	35	M0	41	M2	62	M4	10			
Dynamic events													
Dynamic events	Occurrence time	Event type	Information about dynamic events										
D1	26	0	M1	12									
D2	76	1	J23	M4	28	M3	45	M1	17	M0	73	M2	38
D3	119	1	J24	M0	65	M1	35	M3	50	M2	29	M4	31
D4	236	1	J25	M2	23	M1	49	M3	75	M0	92	M4	26
D5	0	2	J3	M2	27								
D6	0	2	J8	M1	60								

operating system and run on a laptop with Intel Pentium Core i5, 2.50 GHz, 8 GB RAM. Finally, the numerical experiments are conducted.

After proving the performance of our proposed method, the performance of proposed methods (GAKK, GASPT, GALPT, GASRPT, GALRPT, GAM) are compared with conventional GA and TS algorithms using dynamic job shop scheduling test problems. For each problem, the proposed methods are run 5 times and the best results that give a minimum completion time are accepted as the

solution. These solutions are summarized in Table 7. In this table, the best fitness values (C_{max}) are obtained by each method, their CPU times in terms of seconds, and the deviations of each method related to the best solution are presented. As an example, the data, solution and Gantt chart of a 5×5 dynamic job shop scheduling problem with GAM method are given in Appendix A in Tables A.1 and A.2 and Fig. A.1 respectively.

Table 7 shows that the GAKK method provides better solutions than GA in terms of both results and CPU times. For 8×5 and

Table A.13
Data of 12 × 10 dynamic job scheduling problem.

Machine sequences and processing times																					
Jobs	Machine sequences and processing times																				
J1	M7	8	M9	75	M0	72	M6	74	M4	30	M8	43	M2	38	M5	98	M1	26	M3	19	
J2	M6	19	M8	73	M3	43	M0	23	M1	85	M4	39	M5	13	M9	26	M2	67	M7	9	
J3	M1	50	M3	93	M5	80	M4	7	M0	55	M2	61	M6	57	M8	72	M9	42	M7	46	
J4	M1	68	M7	43	M4	99	M6	60	M5	68	M0	91	M8	11	M3	96	M9	11	M2	72	
J5	M7	84	M2	34	M8	40	M5	7	M1	70	M6	74	M3	12	M0	43	M9	69	M4	30	
J6	M8	60	M0	49	M4	59	M5	72	M9	63	M1	69	M7	99	M6	45	M3	27	M2	9	
J7	M6	71	M2	91	M8	65	M1	90	M9	98	M4	8	M7	50	M0	75	M5	37	M3	17	
J8	M8	62	M7	90	M5	98	M3	31	M2	91	M4	38	M9	72	M1	9	M0	72	M6	49	
J9	M4	35	M0	39	M9	74	M5	25	M7	47	M3	52	M2	63	M8	21	M6	35	M1	80	
J10	M9	58	M0	5	M3	50	M8	52	M1	88	M6	20	M2	68	M5	24	M4	53	M7	57	
J11	M7	99	M3	91	M4	33	M5	19	M2	18	M6	38	M0	24	M9	35	M1	49	M8	9	
J12	M0	68	M3	60	M2	77	M7	10	M8	60	M5	15	M9	72	M1	18	M6	90	M4	18	
Dynamic events																					
Dynamic events		Occurrence time		Event type		Information about dynamic events															
D1		28		0		M8 7															
D2		185		1		J13 M5 76 M7 44 M0 16 M1 8 M9 24 M6 12 M4 97 M2 81 M8 78 M3 20															
D3		0		2		J3 M4 12															
D4		0		2		J9 M5 30															
D5		0		2		J3 M0 61															

Table A.14
Data of 13 × 10 dynamic job scheduling problem.

Machine sequences and processing times																					
Jobs	Machine sequences and processing times																				
J1	M7	8	M9	75	M0	72	M6	74	M4	30	M8	43	M2	38	M5	98	M1	26	M3	19	
J2	M6	19	M8	73	M3	43	M0	23	M1	85	M4	39	M5	13	M9	26	M2	67	M7	9	
J3	M1	50	M3	93	M5	80	M4	7	M0	55	M2	61	M6	57	M8	72	M9	42	M7	46	
J4	M1	68	M7	43	M4	99	M6	60	M5	68	M0	91	M8	11	M3	96	M9	11	M2	72	
J5	M7	84	M2	34	M8	40	M5	7	M1	70	M6	74	M3	12	M0	43	M9	69	M4	30	
J6	M8	60	M0	49	M4	59	M5	72	M9	63	M1	69	M7	99	M6	45	M3	27	M2	9	
J7	M6	71	M2	91	M8	65	M1	90	M9	98	M4	8	M7	50	M0	75	M5	37	M3	17	
J8	M8	62	M7	90	M5	98	M3	31	M2	91	M4	38	M9	72	M1	9	M0	72	M6	49	
J9	M4	35	M0	39	M9	74	M5	25	M7	47	M3	52	M2	63	M8	21	M6	35	M1	80	
J10	M9	58	M0	5	M3	50	M8	52	M1	88	M6	20	M2	68	M5	24	M4	53	M7	57	
J11	M7	99	M3	91	M4	33	M5	19	M2	18	M6	38	M0	24	M9	35	M1	49	M8	9	
J12	M0	68	M3	60	M2	77	M7	10	M8	60	M5	15	M9	72	M1	18	M6	90	M4	18	
J13	M9	79	M1	60	M3	56	M6	91	M2	40	M8	86	M7	72	M0	80	M5	89	M4	51	
Dynamic events																					
Dynamic events		Occurrence time		Event type		Information about dynamic events															
D1		28		0		M8 7															
D2		185		1		J14 M5 76 M7 44 M0 16 M1 8 M9 24 M6 12 M4 97 M2 81 M8 78 M3 20															
D3		0		2		J3 M4 12															
D4		0		2		J9 M5 30															
D5		0		2		J11 M2 23															

10 × 5 problem sizes only, the same results are obtained with GAKK and GA. Especially for large scale problems, better solutions are obtained with GASPT, GALPT, GASRPT, GALRPT methods than conventional GA. It is not possible to make a ranking between GAKK, GASPT, GALPT, GASRPT, GALRPT methods in terms of solution quality and CPU times as the best method varies according to the problem. So in this paper, a GAM method is proposed in which 25% of initial population is generated with a 5% mixture of KK, SPT, LPT, SRPT, LRPT + swap. GAM method provides better solutions than other methods for all problem instances. Also, the proposed hybrid methods have shorter CPU times than GA for all problem instances. On the other hand, it is obvious that CPU times increase when the size of the problem increases. As the schedule is rescheduled each time a dynamic event occurs, the CPU time is also

affected by the number of dynamic events. So, this situation should be considered in the evaluation of CPU times.

In Table 7, deviations of the results from the GAM method, which provides the best solution for each problem size, are also given. According to these values, GAM provides better results by 0–254.77% when compared to the GA method. On the other hand, a conventional TS algorithm compared to proposed methods offers the worse results in terms of solution quality. The deviation of GA from the GAM method increases dramatically as the problem sizes gets bigger. This means the GAM method provides superior solutions than GA especially in large-scale problems. GAM provides better results by 0–14.92% compared to GAKK, by 0–153.28% compared to GASPT, by 0 – 148.13% compared to GALPT, by 0–15.65% compared to GASRPT, and by 0–10.07% compared to GALRPT.

Table A.15Data of 20×7 dynamic job scheduling problem.

Machine sequences and processing times																		
Jobs	Machine sequences and processing times																	
J1	M6	71	M2	34	M1	21	M5	95	M4	21	M0	53	M3	55				
J2	M4	55	M5	98	M3	39	M0	12	M6	77	M2	31	M1	42				
J3	M5	37	M4	92	M2	64	M6	54	M1	19	M0	83	M3	34				
J4	M1	87	M5	77	M0	93	M3	69	M2	87	M6	41	M4	60				
J5	M2	98	M5	25	M6	75	M1	49	M3	17	M0	44	M4	96				
J6	M1	7	M4	61	M0	95	M2	35	M5	28	M3	76	M6	9				
J7	M5	59	M0	46	M4	28	M6	52	M3	16	M2	59	M1	91				
J8	M5	9	M4	20	M6	54	M3	41	M0	87	M1	45	M2	39				
J9	M1	28	M0	78	M2	37	M3	26	M5	33	M6	89	M4	33				
J10	M4	96	M3	27	M6	78	M5	84	M2	94	M1	74	M0	69				
J11	M4	24	M2	17	M3	87	M5	76	M6	18	M1	31	M0	20				
J12	M5	28	M1	72	M2	23	M3	99	M6	76	M4	45	M0	58				
J13	M2	17	M4	98	M3	48	M1	46	M6	67	M0	42	M5	27				
J14	M0	80	M3	19	M5	28	M2	50	M4	94	M6	63	M1	12				
J15	M0	75	M4	61	M6	37	M2	50	M5	14	M3	55	M1	41				
J16	M3	96	M2	14	M5	57	M0	47	M4	75	M1	71	M6	60				
J17	M1	31	M3	32	M4	44	M5	58	M6	34	M0	33	M2	69				
J18	M1	44	M2	17	M0	62	M6	15	M3	29	M5	8	M4	97				
J19	M2	58	M3	50	M4	63	M0	57	M6	21	M1	39	M5	20				
J20	M1	85	M0	84	M5	56	M3	61	M2	90	M6	67	M4	20				
Dynamic event																		
Dynamic events	Occurrence time	Event type	Information about dynamic events															
D1	74	0	M4	4														
D2	296	0	M5	7														
D3	720	1	J21	M1	34	M3	61	M5	80	M0	21	M2	79	M4	18	M6	87	
D4	1010	1	J22	M6	33	M2	14	M0	7	M4	6	M1	31	M3	65	M5	9	
D5	0	2	J13	M2	22													

Table A.16Data of 15×10 dynamic job scheduling problem.

Machine sequences and processing times																											
Jobs	Machine sequences and processing times																										
J1	M2	34	M3	55	M5	95	M9	16	M4	21	M6	71	M0	53	M8	52	M1	21	M7	26							
J2	M3	39	M2	31	M0	12	M1	42	M9	79	M8	77	M6	77	M5	98	M4	55	M7	66							
J3	M1	19	M0	83	M3	34	M4	92	M6	54	M9	79	M8	62	M5	37	M2	64	M7	43							
J4	M4	60	M2	87	M8	24	M5	77	M3	69	M7	38	M1	87	M6	41	M9	83	M0	93							
J5	M8	79	M9	77	M2	98	M4	96	M3	17	M0	44	M7	43	M6	75	M1	49	M5	25							
J6	M8	35	M7	95	M6	9	M9	10	M2	35	M1	7	M5	28	M4	61	M0	95	M3	76							
J7	M4	28	M5	59	M3	16	M9	43	M0	46	M8	50	M6	52	M7	27	M2	59	M1	91							
J8	M5	9	M4	20	M2	39	M6	54	M1	45	M7	71	M0	87	M3	41	M9	43	M8	14							
J9	M1	28	M5	33	M0	78	M3	26	M2	37	M7	8	M8	66	M6	89	M9	42	M4	33							
J10	M2	94	M5	84	M6	78	M9	81	M1	74	M3	27	M8	69	M0	69	M7	45	M4	96							
J11	M1	31	M4	24	M0	20	M2	17	M9	25	M8	81	M5	76	M3	87	M7	32	M6	18							
J12	M5	28	M9	97	M0	58	M4	45	M6	76	M3	99	M2	23	M1	72	M8	90	M7	86							
J13	M5	27	M9	48	M8	27	M7	62	M4	98	M6	67	M3	48	M0	42	M1	46	M2	17							
J14	M1	12	M8	50	M0	80	M2	50	M9	80	M3	19	M5	28	M6	63	M4	94	M7	98							
J15	M4	61	M3	55	M6	37	M5	14	M2	50	M8	79	M1	41	M9	72	M7	18	M0	75							
Dynamic events																											
Dynamic events		Occurrence time		Event type		Information about dynamic events																					
D1		28		0		M3		7																			
D2		245		0		M7		5																			
D3		75		1		J16		M6	52	M8	24	M2	35	M0	47	M1	38	M7	41	M9	18	M4	20	M3	45	M5	78
D4		146		1		J17		M3	18	M4	95	M2	61	M8	34	M9	64	M6	5	M7	25	M5	72	M1	92	M0	19
D5		0		2		J3		M6 62																			

According to the results, it is easy to conclude that the GAM method provides better solutions than other methods for all problem instances in terms of both solution quality and CPU time. In Figs. 9 and 10, solution graphs of a 10×10 DJSS problem with GA and GAM are given respectively. In these graphs, it is clear that the GAM method provides better solutions than GA in terms of solution quality and CPU time. All these results indicate that the generation of 25% of the initial solution with a mixture of KK, SPT, LPT, SRPT and LRPT + swap improves the efficiency of proposed methods. In this way, GAM starts with a better initial

solution and this leads to the generation of individuals that have better fitness values for each generation.

6. Conclusions

Most publications in the job scheduling area focus on static scheduling problems and seldom take into account dynamic factors. In order to contribute to the literature, this paper presents a dynamic job shop scheduling problem in which new job arrival,

Table A.17
Solution of 20×5 job scheduling problem (LA14) with GAM.

Machines	M0	M1	M2	M3	M4	J10	J2	J7	J11	J5	J3	J19	J15	J18	J16	J1	J6	J9	J12	J4	J14	J13	J17	J20	J8
Job Sequence																									
Start Time						0	96	117	210	305	388	400	469	556	589	609	662	754	798	859	914	942	988	1066	1162
Finish Time						96	117	210	305	388	400	469	556	589	609	662	754	798	859	914	942	988	1066	1162	1222
Job Sequence						J13	J18	J16	J10	J1	J17	J14	J9	J20	J3	J11	J8	J2	J5	J5	J7	J12	J15	J6	J4
Start Time						0	91	180	234	309	404	432	484	533	611	653	660	701	772	791	865	952	961	1006	1060
Finish Time						91	180	234	309	330	432	484	533	611	653	660	701	772	791	865	952	961	1006	1060	1137
Job Sequence						J4	J1	J14	J13	J20	J12	J6	J19	J10	J7	J3	J18	J8	J15	J9	J5	J16	J11	J17	J2
Start Time						0	66	100	127	336	381	476	519	613	656	743	774	820	820	859	957	1021	1092	1127	1164
Finish Time						66	100	127	186	381	476	519	613	656	743	774	820	820	859	957	1021	1092	1127	1164	1190
Job Sequence						J4	J8	J11	J2	J17	J5	J15	J20	J16	J1	J6	J18	J19	J10	J7	J14	J9	J12	J3	J13
Start Time						0	176	305	381	433	518	556	611	683	697	754	816	882	909	988	1057	1107	1124	1159	1198
Finish Time						1430	200	381	433	459	552	597	680	697	752	816	882	909	988	1057	1107	1124	1159	1198	1214
Job Sequence						J19	J15	J8	J6	J20	J9	J12	J17	J7	J5	J4	J14	J16	J13	J2	J10	J11	J3	J18	J1
Start Time						0	84	93	176	255	336	361	371	404	481	518	597	640	683	772	788	865	893	991	1030
Finish Time						84	93	176	255	336	361	371	404	481	518	597	640	683	742	788	865	893	991	1033	1128

machine breakdown and change in processing time occur. Since dynamic job shop scheduling problems are NP-hard combinatorial optimization problems, heuristic methods are useful for solving these kinds of problems. This paper introduces hybrid GA methodologies, where a new KK heuristic + swap and well-known dispatching rules + swap are integrated with a GA algorithm to provide fast and efficient solutions for large size dynamic job shop scheduling problems. In this way, proposed methods deliver outstanding solutions for the dynamic job scheduling problem in terms of solution quality and CPU time.

This paper also presents a comprehensive benchmark platform for the comparison of models and algorithms with applications for dynamic job shop scheduling problems. It provides a scalable test environment which reflects many important features of dynamic job shop scheduling such as machine breakdown, new job arrival, and change in processing time. Various test instances have been generated and solved by powerful heuristics which can serve as a benchmark for future research in the field.

As the proposed methods take dynamic factors into account and provide efficient solutions even in large scale problems, they can also be applied to real systems. In future studies, proposed methods may be applied to different job scheduling environments such as parallel machines, flow shop, flexible job shop and open shop. Maximum completion time (makespan) is considered as a performance measure. Different performance measures can be used in future studies.

Acknowledgements

This work was completely supported by the Department of Scientific Research Projects, PAU under the project number 2011SOBE036.

Appendix A

In this table event type “0” indicates machine breakdown Table A.1. For instance, dynamic event D1 means machine M1 breakdowns on the 12th day and it continues for 3 days. Event type “1” indicates new job arrival. For example, dynamic event D3 means, a new job J6 arrives on the 5th day and this job's job sequence on the machines is M2, M4, M1, M3, M0 and processing times on each machine are 7, 3, 6, 1, 4 respectively. Event type “2” indicates change in the processing time of an operation. For instance, dynamic event D5 means the processing time of job J4 on machine M0 changes as 6 days. Using the data on A.1 the problem is solved with GAM method. The result that is obtained with GAM and its Gantt chart can be seen in the appendix between Table A.2 and Fig. A.1 respectively (see Fig. A.2 and Tables A.3–A.17).

References

- Adibi, M. A., Zandieh, M., & Amiri, M. (2010). Multi-objective scheduling of dynamic job shop using variable neighborhood search. *Expert Systems with Applications*, 37, 282–287.
- Balas, E., & Vazacopoulos, A. (1998). Guided local search with shifting bottleneck for job shop scheduling. *Management Science*, 44(2), 262–275.
- Barnes, J. W., & Chambers, J. B. (1995). Solving the job shop scheduling problem using tabu search. *IIE Transactions*, 27, 257–263.
- Binato, S., Hery, W. J., Loewenstern, D. M., & Resende, M. G. C. (2002). A GRASP for job shop scheduling. *Essays and Surveys in Metaheuristics*, 59–79.
- Chang, F. C. R. (1997). Heuristics for dynamic job shop scheduling with real-time updated queueing time estimates. *International Journal of Production Research*, 35(3), 651–665.
- Church, L. K., & Uzsoy, R. (1992). Analysis of periodic and event-driven rescheduling policies in dynamic shops. *International Journal of Computer Integrated Manufacturing*, 5(3), 153–163.
- Dell'Amico, M., & Trubian, M. (1993). Applying tabu-search to job-shop scheduling problem. *Annals of Operations Research*, 41(1–4), 231–252.

- Dominic, P. D. D., Kaliyamoorthy, S., & Kumar, S. (2004). Efficient dispatching rules for dynamic job shop scheduling. *International Journal of Advanced Manufacturing Technology*, 24, 70–75.
- Fattahi, P., & Fallahi, A. (2010). Dynamic scheduling in flexible job shop systems by considering simultaneously efficiency and stability. *CIRP Journal of Manufacturing Science and Technology*, 2, 114–123.
- Fattahi, P., Mehrabad, M. S., & Jolai, F. (2007). Mathematical modelling and heuristic approaches to flexible job shop scheduling problems. *Journal of Intelligent Manufacturing*, 18, 331–342.
- Fisher, H., & Thompson, G. L. (1963). Probabilistic learning combinations of local job-shop scheduling rules. In J. F. Muth & G. L. Thompson (Eds.), *Industrial scheduling* (pp. 225–251). Englewood Cliffs, New Jersey: Prentice Hall.
- Gao, Y., Ding, Y. S., & Zhang, H. Y. (2009). Job-shop scheduling considering rescheduling in uncertain dynamic environment. In *16th international conference on management science & engineering, Moscow, Russia* (pp. 380–384).
- Ge, H., Du, W., & Qian, F. (2007). A hybrid algorithm based on particle swarm optimization and simulated annealing for job shop scheduling. In *Proceedings of the 3rd international conference on natural computation* (pp. 715–719) Haikou, China.
- Ge, H., Sun, L., Liang, Y., & Qian, F. (2008). An effective PSO and AIS-based hybrid intelligent algorithm for job-shop scheduling. *IEEE Transactions on Systems, Man and Cybernetics, Part A*, 38(2), 358–368.
- Gendreau, M., & Potvin, Y. J. (2010). Tabu search. In M. Gendreau & Y. J. Potvin (Eds.), *Handbook of metaheuristics* (2nd ed., pp. 41–59). New York, USA: Springer.
- Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5), 533–549.
- Goldberg, D. E. (1989). *Genetic algorithms in search, optimization and machine learning*. USA: Addison Wesley Longman.
- Goncalves, J., Magalhaes Mendes, J., & Mauricio, M. G. C. (2005). A hybrid genetic algorithm for the job shop scheduling problem. *European Journal of Operational Research*, 167(1), 77–95.
- Hao, X. C., & Lin, L. (2010). Job shop rescheduling by using multi-objective genetic algorithm. In *The 40th international conference on computers and industrial engineering (CIE)* (pp. 1–6).
- Hao, X. C., & Gen, M. (2011). Multi-objective job shop rescheduling by using evolutionary algorithm. *IEEJ Transactions on Electronics, Information and Systems*, 131(3), 674–681.
- Holloway, C. A., & Nelson, R. T. (1974). Job shop scheduling with due dates and variable processing times. *Management Science*, 20(9), 1264–1275.
- Hosseinabadi, A. A. R., Siar, H., Shamshirband, S., Shojafar, M., & Nasir, M. H. N. M. (2014). Using the gravitational emulation local search algorithm to solve the multi-objective flexible dynamic job shop scheduling problem in Small and Medium Enterprises. *Annals of Operations Research*, 229, 451–474 (2015).
- Joseph, A., Egon, B., & Daniel, Z. (1988). The shifting bottleneck procedure for job shop scheduling. *Management Science*, 34(3), 391–401.
- Kapanoglu, M., & Alikalfa, M. (2011). Learning IF–THEN priority rules for dynamic job shops using genetic algorithms. *Robotics and Computer-Integrated Manufacturing*, 27, 47–55.
- Kulak, O., Sahin, Y., & Taner, M. E. (2012). Joint order batching and picker routing in single and multiple-cross-aisle warehouses using cluster-based tabu search algorithms. *Flexible Services and Manufacturing Journal*, 24, 52–80.
- Kulak, O., Yilmaz, I. O., & Günther, H. O. (2007). PCB assembly scheduling for collect-and-place machines using genetic algorithms. *International Journal of Production Research*, 45, 3949–3969.
- Kutanoglu, E., & Sabuncuoglu, I. (2001). Routing-based reactive scheduling policies for machine failures in dynamic job shops. *International Journal of Production Research*, 39(14), 3141–3158.
- Lawrence, S. (1984). *Resource constrained project scheduling: An experimental investigation of heuristic scheduling techniques (Supplement)*. Pittsburgh, Pennsylvania: Graduate School of Industrial Administration, Carnegie-Mellon University.
- Li, Y., & Chen, Y. (2009). Neural network and genetic algorithm-based hybrid approach to dynamic job shop scheduling problem. In *Proceedings of the 2009 IEEE international conference on systems, man, and cybernetic* (pp. 4836–4841) San Antonio, TX, USA.
- Lin, T., Horng, S., Kao, T., Chen, Y., & Run, R. (2010). An efficient job-shop scheduling algorithm based on particle swarm optimization. *Expert Systems with Applications*, 37(3), 2629–2636.
- Liu, S. Q., Ong, H. L., & Ng, K. M. (2005). Metaheuristics for minimizing the makespan of the dynamic shop scheduling problem. *Advances in Engineering Software*, 36, 199–205.
- Lu, M.-S., & Romanowski, R. (2013). Multicontextual dispatching rules for job shops with dynamic job arrival. *International Journal of Advanced Manufacturing Technology*, 67, 19–33.
- Mehta, S. V., & Uzsoy, R. (1999). Predictable scheduling of a single machine subject to breakdowns. *International Journal of Computer Integrated Manufacturing*, 12(1), 15–38.
- Motaghedi-larijani, A., Sabri-laghaie, K., & Heydar, M. (2010). Solving flexible job shop scheduling with multi objective approach. *International Journal of Industrial Engineering & Production Research*, 21(4), 197–209.
- Muhlemann, A. P., Lockett, A. G., & Farn, C. K. (1982). Job shop scheduling heuristics and frequency of scheduling. *International Journal of Production Research*, 20(2), 227–241.
- Nelson, R. T., Holloway, C. A., & Wong, R. M. (1977). Centralized scheduling and priority implementation heuristics for a dynamic job shop model. *AIIE Transactions*, 9(1), 95–102.
- Nguyen, S., Zhang, M., Johnston, M., & Tan, K. C. (2014). Automatic design of scheduling policies for dynamic multi-objective job shop scheduling via cooperative coevolution genetic programming. *IEEE Transactions on Evolutionary Computation*, 18(2), 193–208.
- Nie, L., Gao, L., Li, P., & Shao, X. (2013). Reactive scheduling in a job shop where jobs arrive over time. *Computers & Industrial Engineering*, 66, 389–405.
- Nowicki, N., & Smutnicki, C. (1996). A fast taboo search algorithm for the job shop. *Management Science*, 42(6), 797–813.
- Pezzella, F., & Merelli, E. (2000). A tabu search method guided by shifting bottleneck for the job shop scheduling problem. *European Journal of Operational Research*, 120(2), 297–310.
- Pinedo, M. L. (2008). *Scheduling theory algorithms and systems*. USA: Springer.
- Qi, J. G., Burns, G. R., & Harrison, D. K. (2000). The application of parallel multipopulation genetic algorithms to dynamic job-shop scheduling. *International Journal of Advanced Manufacturing Technology*, 16, 609–615.
- Qing-dao-er-ji, R., & Wang, Y. (2012). A new hybrid genetic algorithm for job shop scheduling problem. *Computers & Operations Research*, 39, 2291–2299.
- Qiu, X., & Lau, H. Y. K. (2013). An AIS-based hybrid algorithm with PDRs for multi-objective dynamic online job shop scheduling problem. *Applied Soft Computing*, 13, 1340–1351.
- Rajendran, C., & Holthaus, O. (1999). A comparative study of dispatching rules in dynamic flowshops and jobshops. *European Journal of Operational Research*, 116, 156–170.
- Rangsaritratsamee, R., Ferrell, W., Jr., & Kurz, M. (2004). Dynamic rescheduling that simultaneously considers efficiency and stability. *Computers & Industrial Engineering*, 46, 1–15.
- Sabuncuoglu, I., & Bayiz, M. (2000). Analysis of reactive scheduling problems in a job shop environment. *European Journal of Operational Research*, 126, 567–586.
- Sharma, P., & Jain, A. (2014). Analysis of dispatching rules in a stochastic dynamic job shop manufacturing system with sequence-dependent setup times. *Frontiers of Mechanical Engineering*, 9(4), 380–389.
- Suwa, H., & Sandoh, H. (2007). Capability of cumulative delay based reactive scheduling for job shops with machine breakdowns. *Computers & Industrial Engineering*, 53, 63–78.
- Van Laarhoven, P. J. M., Aarts, E. H. L., & Lenstra, J. K. (1992). Job shop scheduling by simulated annealing. *Operational Research*, 40(1), 113–125.
- Vieira, G. E., Hermann, J. W., & Lin, E. (2003). Rescheduling manufacturing systems: A framework of strategies, policies and methods. *Journal of Scheduling*, 6(1), 36–92.
- Vinod, V., & Sridharan, R. (2008). Scheduling a dynamic job shop production system with sequence dependent setups: An experimental study. *Robotics and Computer Integrated Manufacturing*, 24, 435–449.
- Wang, X., & Duan, H. (2014). A hybrid biogeography-based optimization algorithm for job shop scheduling problem. *Computers & Industrial Engineering*, 73, 96–114.
- Zhang, L., Gao, L., & Li, X. (2013). A hybrid genetic algorithm and tabu search for multi-objective dynamic job shop scheduling problem. *International Journal of Production Research*, 51(12), 3516–3531.
- Zhang, C. Y., Li, P. G., Guan, Z. L., & Rao, Y. Q. (2007). A tabu search algorithm with a new neighborhood structure for the job shop scheduling problem. *Computers & Operations Research*, 34, 3229–3242.
- Zhou, R., Nee, A. Y. C., & Lee, H. P. (2009). Performance of an ant colony optimization algorithm in dynamic job shop scheduling problems. *International Journal of Production Research*, 47(11), 2903–2920.